

Calcolo Parallelo

Moreno Manzolani

i supercomputer di oggi saranno i telefoni di domani

- > Cosa sono archi paralleli
- > Programmare e creare programmi paralleli
 - > da GPU $\rightarrow$ supercomputer
- > Misurare prestazioni ed efficienza

PREREQUISITI

- Programmazione C
- Ambiente Linux / Unix $\rightarrow$ * muoversi nel cluster ssh

Ripasso su

- Architettura degli Elaboratori
- Algoritmi e Strutture dati
- Teoria delle Concorrenze Deadlock, Race Conditions

ESAME

- Written exam (weight: 60%) 1.5h.
 - In-person, closed book
 - Open questions and/or simple exercises on all topics discussed during the course
 - You can refuse the grade and retry the written exam
- Individual programming project + written report (weight: 40%)
 - Specification defined by the instructor; the same project for the whole academic year
 - Project can be handed in at any time before Feb 28, 2027
 - There is no discussion, unless I request it
 - If you refuse the grade, you must hand-in a NEW project on NEW specifications
- Final grade rounded to the nearest integer
- The written exam and programming project are independent, and can be done in any order

Saranno esercizi simili a quelli visti in classe

lo stesso per tutti $\rightarrow$ 1 all'anno

per cose non chiare

Perché esiste il calcolo parallelo?

- > molte app richiedono molta potenza di calcolo $\rightarrow$ Previsioni Meteo (requisiti di tempo impiegato)
" + accuratezza "
- $\rightarrow$ Simulazioni climatiche - a livello globale su larghe scale di tempo
- $\rightarrow$ Animazioni 3D
- $\rightarrow$ AI e ...

x risolvere problemi + complessi || in meno tempo || meno uso delle risorse

HPC: Parallel Computing

$L = \exists!$ app che prende molti dati di INPUT $\rightarrow$ da dividere in sottoproblemi indipendenti $\pm$ x arrivare alla sol finale

- > hardware proprietario: Reti di interconnessioni || CPU
- single app in meno tempo possibile
 - $\hookrightarrow$ 1 o pochi task

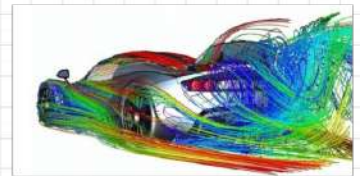
HTC: Cloud Computing

- $\hookrightarrow$ tante istanze indipendenti
- $\hookrightarrow$ hardware commodity

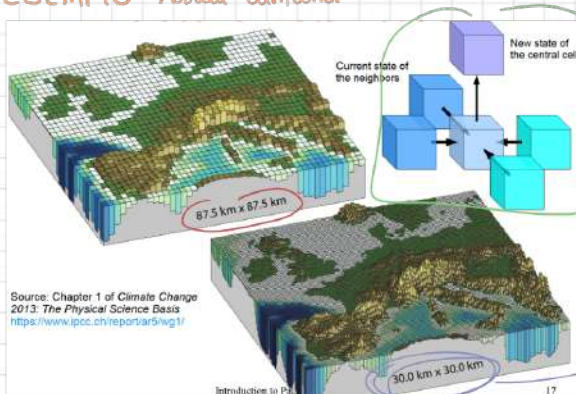
la stessa app $\rightarrow$ + task nel - tempo possibile

ESEMPIO Numerical Wind Tunnel

crash test di supercar $\rightarrow$ modelli matematici $\rightarrow$ per simulare fluido dinamica
 $\hookrightarrow$ molte aziende hanno un loro quasi supercomputer



ESEMPIO Modelli Climatici



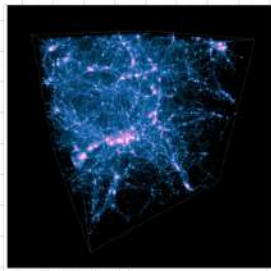
Quale sarà la previsione climatica, la situazione in quella cella?

ogni cella calcola la nuova situazione in base ai vicini

Se riduco di $\frac{1}{3} \Rightarrow$ aumenta di $3^2 = 9 \Rightarrow$ molti + dati da elaborare
+ potenza

riducendo la granularità $\Rightarrow$ migliore risultati

ESEMPIO Modello Cosmologico



"The Illustris simulations were run on supercomputers in France, Germany, and the US. The largest was run on 8,192 compute cores, and took 19 million GPU hours"



Most of the Illustris simulation was executed in the Curie supercomputer with 440,000 x86 cores in nearly 5,000 nodes, 22 petaflop/s peak computing power

for each particle p i
compute the force on p by all other
particles q ;
compute the new position of p using
Newton's second law;

applichiamo le leggi delle fisica
x simulare

→ distr. delle materia nell'universo, ma su scale piccole
vedo che tende ad addensarsi in filamenti

si applica bene al Calcolo Parallelo ⇒ si applica la 2^a legge di Newton

tipo di calcolo parallelizzabile in modo efficiente
 $O(n \log n)$

In passato si faceva affidamento alle Legge di Moore: "Num transistor raddoppia ogni 24 mesi"

→ Corollario: prestazioni dei microprocessori raddoppiano ogni 2 anni

⇒ Nel mondo reale nulla può crescere esponenzialmente all'∞

↓
Il calore prodotto è esponenziale

Power

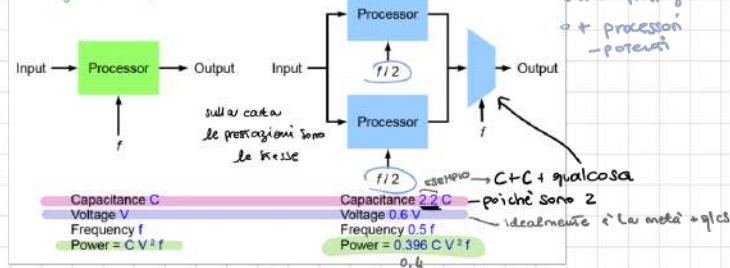
The power required by an IC (e.g., a processor) can be expressed as

$$\text{Power} = C \times V^2 \times f$$

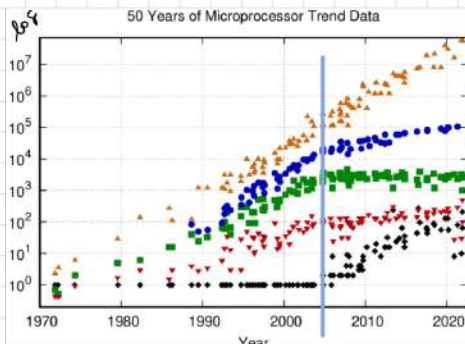
where:

- C is the capacitance (ability of a circuit to store energy)
- V is the voltage
- f is the frequency at which the processor operates

Design Convergente



In realtà consuma il 40% dell'altro ⇒ + calore



Arch Parallela

continua ad aumentare

appiattiva

leggero trend ↑

tanti core in uno stesso chip

→ questi core consentono di aumentare le prestazioni + core dalla stessa potenza

La miniaturizzazione non è ∞ ma c'è ancora spazio

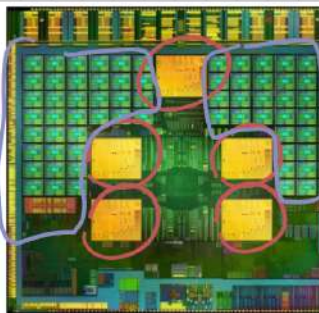
HW designers create processors with more cores

Result:

- parallel hardware is ubiquitous
- parallel software is rare

The challenge

- Make parallel software as common as parallel hardware



NVIDIA Tesla 4 SoC

Core CPU
GPU

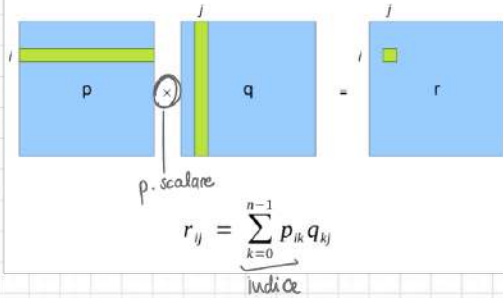
→ qualcuno progetterà questo hardware

impareremo a sfruttarlo, x orchestrare al meglio lo scambio di dati

Parallel programming in brief

- Decompose** the problem in sub-problems
- Distribute** sub-problems to the available execution units
- Solve** sub-problems
 - Cooperate to solve sub-problems, if necessary
- Combine** the partial results
- Goals**
 - Reduce the wall-clock time
 - Balance the workload across execution units
 - Reduce communication and synchronization overhead

Matmul



x ridurre tempi di inferenze
in sys basati sull'AI

c'è divide et impera — ma —> difficile da implementare efficacemente

Hardware

Feature	Value
Processor type	Intel(R) Core(TM) i9-12900F
Clock frequency	5.1 GHz
Processor chips	1
Processing cores	8 P-core + 8 E-core
Hyperthreading	2 way (→ 16 virtual P-cores)
Floating-point unit	614 GFlops
L1-cache	640 KB
L1-dcache	768 KB
L2-cache	14 MB
L3-cache	30 MB
DRAM	64 GB
OS	Ubuntu Linux 24.04
Python	3.12.3
gcc	Ubuntu 13.3.0-6ubuntu2-24.04
Java	OpenJDK 21.0.10 2026-01-20

potenze di picco — Quanti di questi riusciremo ad usare in pratica?

! tutte le info dell'hw che si usa

1 implementazione Python Pura

```
# matmul-test.py
import sys, random

n = 4096 non così grande, è normale

p = [[random.random() for row in range(n)]
      for col in range(n)]
q = [[random.random() for row in range(n)]
      for col in range(n)]
r = [[0 for row in range(n)]
      for col in range(n)]

start = time()
for i in range(n):
    for j in range(n):
        for k in range(n):
            r[i][j] += p[i][k] * q[k][j]
elapsed = time() - start
print(elapsed)
```

Running time
= **6711** seconds
= **1.8** hours

$2n^3$ floating-point
operations in
6711 sec
≈ **20 MFLOPS**

Peak = 614
GFLOPS

Python gets
0.00003% of
peak

2×4096^3

Quante operazioni floating point ci interessano?
 $2 \times n^3$

2 implementazione Java

```
import java.util.Random;
import java.io.*;

public class matmul_test {
    final static int n = 4096;
    static float[][] p = new float[n][n];
    static float[][] q = new float[n][n];
    static float[][] r = new float[n][n];

    public static void main(String[] args) {
        Random rnd = new Random();
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j++) {
                p[i][j] = rnd.nextFloat();
                q[i][j] = rnd.nextFloat();
                r[i][j] = 0;
            }
        }

        final long start = System.nanoTime();
        for (int i=0; i<n; i++)
            for (int j=0; j<n; j++)
                for (int k=0; k<n; k++)
                    r[i][j] += p[i][k] * q[k][j];
        final long stop = System.nanoTime();
        System.out.println((stop - start)*1e-9);
    }
}
```

Running time
= **322** seconds
= **5.4** minutes

21 times faster
than Python

← veloce del C non ottimizzato

3 implementazione C

```
void matmul(const float * restrict p,
            const float * restrict q,
            float * restrict r, int n)
{
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            r[i*n + j] = 0;

    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            for (int k=0; k<n; k++)
                r[i*n + j] += p[i*n + k] * q[k*n + j];
}
```

nel programma accederà a p
esclusivamente dalla var p

} Sono aree di memoria separate

ESEMPIO

```
int *a;
int *b;
int *c;
int pippo;

*a = 10;
*b = 5;
*c = 7;

*c = *b
print(*a) → '10' → '7'
```

a, b, c = &pippo

← sembra che cambio valore — ma è solo un alias della stessa area di mem

Running time
= **488** seconds
= **8.1** minutes

14 times faster
than Python

~1.5 times slower
than Java

Note: compiled without
any optimization flag -Ox

	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76
+ -O2 flag	334.08	0.41	0.07%	20.09

- Adding the -O2 flag to gcc reduces the execution time by 30%
 - Still slower than Java!

Vediamo di fare meglio

Notiamo che in questo caso vale

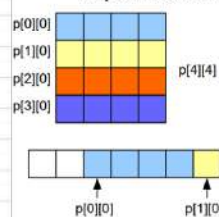
Reordering loops

In this particular examples, the three nested loops can be permuted and still produce the same result

```
for (int i=0; i<n; i++)
  for (int j=0; j<n; j++)
    c[i*n+j] = p[i*n+k] + q[k*n+j];

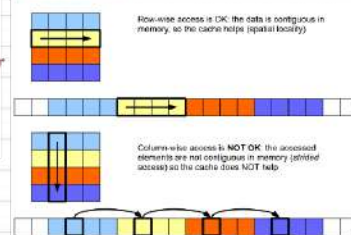
for (int k=0; k<n; k++)
  for (int i=0; i<n; i++)
    for (int j=0; j<n; j++)
      c[i*n+j] += p[i*n+k] * q[k*n+j];
```

- Matrices in C are stored in row-major order
 - Elements of each row are contiguous in memory
 - Adjacent rows are contiguous in memory

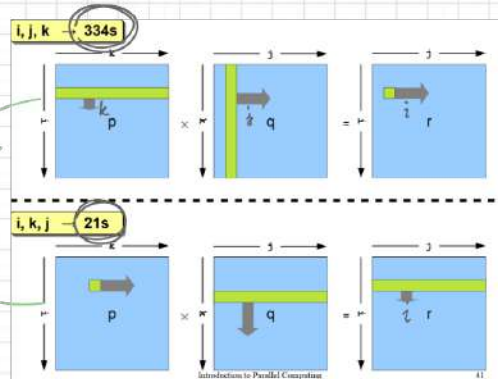


Scorrere una matrice per riga ha più elementi in linea in memoria

x cache line



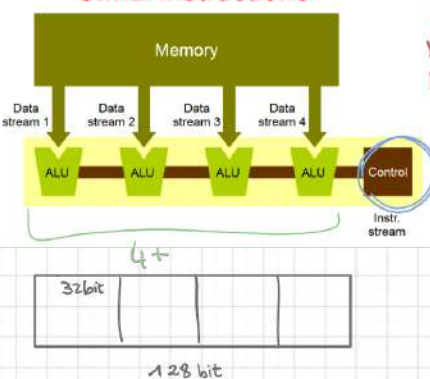
indice che scorre + veloce



forza q ad essere esaminata per colonne

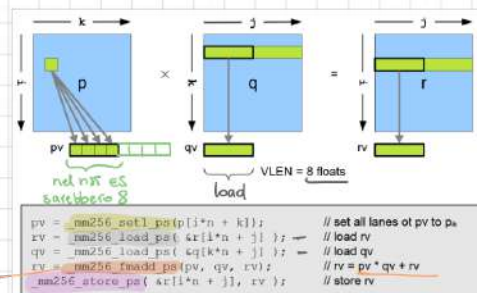
Permutation	Time (s)	GFLOPS
i,j,k	334.08	0.41
i,k,j	20.54	6.69
j,i,k	182.33	0.75
j,k,i	593.29	0.23
k,i,j	21.61	6.36
k,j,i	542.72	0.25

SIMD instructions



Single Instruction Multiple Data

operano contemporaneamente



anche se siamo in C, sembra assembly
→ mappa direttamente ad un'istruzione assembly

4 implementazione C con SIMD

```
void matmul_ijk_simd(const float * restrict p, const float * restrict q, float * restrict r, int n) {
  for (int i=0; i<n; i++) {
    for (int j=0; j<n; j += VLEN) {
      mm256_store_ps(&r[i*n+j], mm256_setzero_ps());
    }
  }

  for (int i=0; i<n; i++) {
    for (int k=0; k<n; k++) {
      const mm256 pv = mm256_broadcast_ss(&p[i*n+k]);
      for (int j=0; j<n; j += VLEN) {
        mm256 rv = mm256_load_ps(&r[i*n+j]);
        const mm256 qv = mm256_load_ps(&q[k*n+j]);
        rv = mm256_fmadd_ps(pv, qv, rv);
        mm256_store_ps(&r[i*n+j], rv);
      }
    }
  }
}
```

perde di leggibilità

sono per arch x86
→ perdo portabilità

codice + lungo
con loop & rolling
riduce num di branch

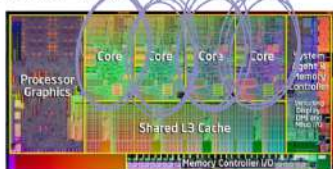
	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76
+ -O2 flag	334.08	0.41	0.07%	20.09
+ Loop reordering	20.54	6.69	1.09%	326.77
+ auto SIMD vettorizzazione autom.	10.15	13.54	2.21%	661.33
+ manual SIMD	12.48	11.01	1.79%	537.64
+ manual SIMD + OpenMP	1.24	110.57	18.01%	5399.72

+ veloce

Multicore parallelism

Modern processors have multiple independent execution units ("cores")

- This machine has 8 of them, each one seen by the OS as two "virtual" cores (Intel HyperThreading)
- Virtual cores share some physical resources and are less powerful than "true" physical cores



```
void matmul_ijk_simd_omp(const float * restrict p, const float * restrict q, float * restrict r, int n) {
  #pragma omp parallel default(none) shared(p,q,r,n)
  {
    #pragma omp for
    for (int i=0; i<n; i++) {
      for (int j=0; j<n; j += VLEN) {
        mm256_store_ps(&r[i*n+j], mm256_setzero_ps());
      }
    }

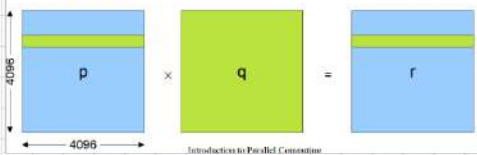
    #pragma omp for
    for (int i=0; i<n; i++) {
      for (int k=0; k<n; k++) {
        const mm256 pv = mm256_broadcast_ss(&p[i*n+k]);
        for (int j=0; j<n; j += VLEN) {
          mm256 rv = mm256_load_ps(&r[i*n+j]);
          const mm256 qv = mm256_load_ps(&q[k*n+j]);
          rv = mm256_fmadd_ps(pv, qv, rv);
          mm256_store_ps(&r[i*n+j], rv);
        }
      }
    }
  }
}
```

ogni core ha i suoi registri SIMD indipendenti

Tiling

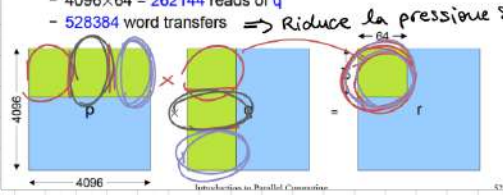
- How many memory accesses are required to compute a row of r ?

- 4096 writes of r
- 4096 reads of p
- 4096 $\times$ 4096 = 16777216 reads of q
- 16785408 word transfers total



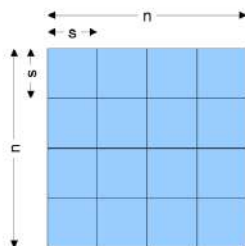
- Instead of computing a row of r (4096 elements), let us compute a 64×64 block (still 4096 elements)

- $64 \times 64 = 4096$ writes of r
- $64 \times 4096 = 262144$ reads of p
- $4096 \times 64 = 262144$ reads of q
- 528384 word transfers $\Rightarrow$ Riduce la pressione sulla mem



```
for (int i=0; i<n; i += s)
  for (int j=0; j<n; j += s)
    for (int k=0; k<n; k += s)
      for (int ii=0; ii<s; ii++)
        for (int jj=0; jj<s; jj++)
          for (int kk=0; kk<s; kk++)
            r[i+ii][j+jj] += p[i+ii][k+kk] * q[k+kk][j+jj];
```

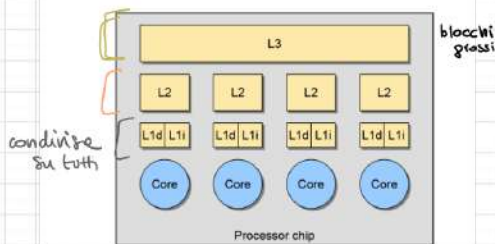
dimensione mattonella = 64



What is the optimal value of s ?

Si ma in QUALE CACHE?

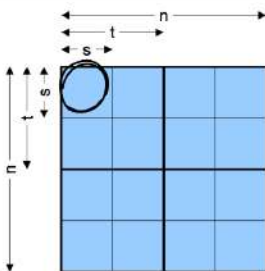
Modern processors have multiple levels of caches



Totamente su tutte le cache

sfruttando tutti i livelli

```
for (int i=0; i<n; i += t)
  for (int j=0; j<n; j += t)
    for (int k=0; k<n; k += t)
      for (int ii=0; ii<t; ii += s)
        for (int jj=0; jj<t; jj += s)
          for (int kk=0; kk<t; kk += s)
            for (int iiii=0; iiii<s; iiii++)
              for (int jjjj=0; jjjj<s; jjjj++)
                for (int kkkk=0; kkkk<s; kkkk++)
                  r[i+ii][j+jj] += p[i+ii][k+kk] * q[k+kk][j+jj];
```



What are the optimal values of s and t ?

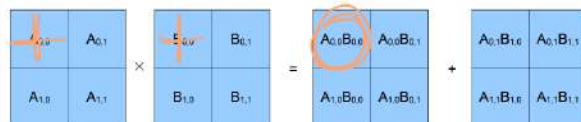
Come scelgo t e s ?

in base al processore $\Rightarrow$ Un CASINO

Usiamo questo che si adatta ed e' meglio

Parallel divide-and-conquer

- Improves data locality and cache usage
- Agnostic with respect to the number of levels of caching and their sizes
- Idea
 - Decompose A and B into four smaller matrices
 - Compute $A \times B$ as sums of eight products
 - Compute the products recursively



Suddivido ulteriormente, ricorrendo

fin quando non sono sufficientemente piccoli

	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76
+ -O2 flag	334.08	0.41	0.07%	20.09
+ Loop reordering	20.54	6.69	1.09%	326.77
+ auto SIMD	10.15	13.54	2.21%	661.33
+ manual SIMD	12.48	11.01	1.79%	537.64
+ manual SIMD + OpenMP	1.24	110.57	18.01%	5399.72
Parallel divide-and-conquer	0.45	308.85	50.30%	15082.80

programmazione multi-core

ASSURDO

il 100% e' praticamente impossibile

caso base

```
void mm_rec( float * restrict C, const float * restrict A, const
float * restrict B, int n, int b) {
  const int THR = 64; // tuning param. */
  if (b <= THR) {
    mm_base(C, A, B, n, b);
  } else {
    #define SUB(M,ro,co) ((M) + ((ro)*n+(co)) * ((b)/2))
    #pragma omp task
    mm_rec(SUB(C,0,0), SUB(A,0,0), SUB(B,0,0), n, b/2);
    #pragma omp task
    mm_rec(SUB(C,0,1), SUB(A,0,0), SUB(B,0,1), n, b/2);
    #pragma omp task
    mm_rec(SUB(C,1,0), SUB(A,1,0), SUB(B,0,0), n, b/2);
    #pragma omp task
    mm_rec(SUB(C,1,1), SUB(A,1,0), SUB(B,0,1), n, b/2);
    #pragma omp taskwait
    #pragma omp task
    mm_rec(SUB(C,0,0), SUB(A,0,1), SUB(B,1,0), n, b/2);
    #pragma omp task
    mm_rec(SUB(C,0,1), SUB(A,0,1), SUB(B,1,1), n, b/2);
    #pragma omp task
    mm_rec(SUB(C,1,0), SUB(A,1,1), SUB(B,1,0), n, b/2);
    #pragma omp task
    mm_rec(SUB(C,1,1), SUB(A,1,1), SUB(B,1,1), n, b/2);
    #pragma omp taskwait
  }
}
```

- The matrix size n must be a power of two.
- Start the recursion with $mm_rec(r, p, q, n, n)$

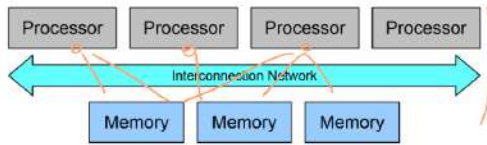
applicazioni del calcolo parallelo

un CAFAN

CON SIMD

Architettura x Calcolo parallelo

20/02/2026



È un'arch. parallela astratta

connessi in qualche modo con le Memorie

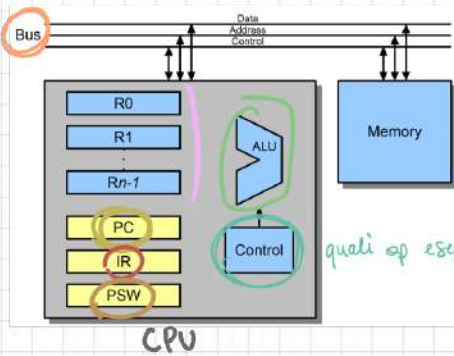
- How is parallelism handled?
- What is the exact physical location of the memories?
- What is the topology of the interconnect network?

Why are parallel architectures important?

- There is no "typical" parallel computer: different vendors use different architectures
- There is currently no "universal" programming paradigm that fits all architectures
 - Parallel programs must be tailored to the underlying parallel architecture
 - The architecture of a parallel computer limits the choice of the programming paradigm that can be used

ci sono framework / algs / linguaggi ...
in base all'arch.

REMINDER Arch. von Neumann



operazioni aritm / logiche sui bit

dimensioni e num. registri variabili

Program Counter

quelli sp. eseguire

Instruction Register - byte dell'istruzione da eseguire

se si verificano condizioni particolari overflow / risultato / eccezioni

Tempi di controllo delle CPU — unità di misura del tempo non percepibile mili / nano secondi

	CPU	Real world
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access	120 ns	6 min
Solid-state disk I/O	50-150 μ s	2-6 days
Rotational disk I/O	1-10 ms	1-12 months
Internet: SF to NYC	40 ms	4 years
Internet: SF to UK	81 ms	8 years
Internet: SF to Australia	183 ms	19 years
Physical system reboot	1 min	6 millennia

Utile rapportarli al mondo reale

3 cicli di clock

9 cicli di clock in attesa dalle cache di 1° livello

prima che un'info arrivi fa "sprecare" tempo che potrebbe essere allocato ad altro

→ ci fa capire che fa tanta differenza di sfruttare bene / male le cache

*fattore moltiplicativo fisso

How to address the bottlenecks of Von Neumann architecture

Reduce the memory access latency

- Rely on CPU registers whenever possible
- Use caches

Hide the memory access latency

- Multithreading and context-switch during memory accesses

Execute multiple instructions at the same time

- Pipelining
- Multiple issue
- Branch prediction
- Speculative execution
- SIMD extensions (ioi)

Necessità di passare ad un'arch. di interpretazione
Processori sono molto più veloci delle RAM

x ridurre le latenze — ottenere + dati dalla CPU x evitare trasferimenti

Esegue trop. + flussi di istruzioni x accedere alla mem

1 CORE fisico visto come 2 virtuali → cos. se uno è bloccato x accesso alla mem l'altro procede

Caching

Processor/Memory wall

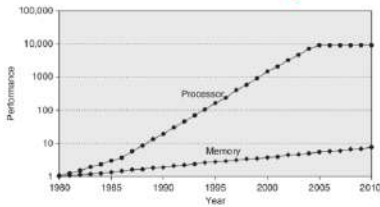


Figure 2.2 Starting with 1980 performance as a baseline, the gap in performance, measured as the difference in the time between processor memory requests (for a single processor or core) and the latency of a DRAM access, is plotted over time. Note that the vertical axis must be on a logarithmic scale to record the size of the processor-DRAM performance gap. The memory baseline is 64 KB DRAM in 1980, with a 1.07 per year performance improvement in latency (see Figure 2.13 on page 99). The processor line assumes a 1.25 improvement per year until 1986, a 1.52 improvement until 2000, a 1.20 improvement between 2000 and 2005, and no change in processor performance (on a per-core basis) between 2005 and 2010; see Figure 1.1 in Chapter 1.

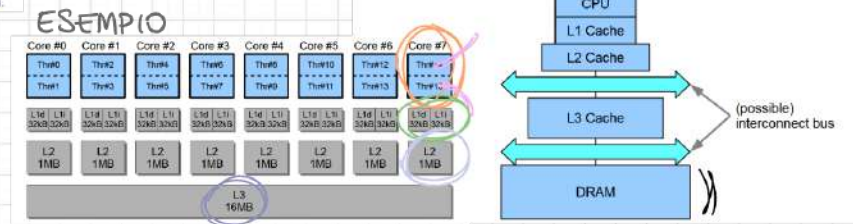
le prestazioni dei processori si stanno livellando (del single-core)
 ↳ ora multi-core ⇒ + complesso per accesso alle mem

di memoria — non sono in grado di offrire un throughput rispetto CORE

Gerarchie di cache

cerchiamo quindi di DIVIDERE le memorie

Large memories are slow; fast memories are small



hyperthreading 4 CORE fisico → 2 CORE indipendenti

l'hardware fa passare il flusso di processi in modo dinamico 1/2 fluss

cache di 1° livello : 2 parti ISTRUZIONI | DATI
 2° livello
 3° livello

Perché le cache migliorano le prestazioni

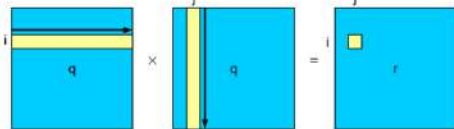
- Cache memory works well when applications exhibit **spatial** and/or **temporal** locality
- Spatial locality**
 - Accessing adjacent memory locations is OK
- Temporal locality**
 - Repeatedly accessing the same memory location(s) is OK

CPU accede alle stesse info probabilmente ad un breve lasso di tempo

ESEMPIO

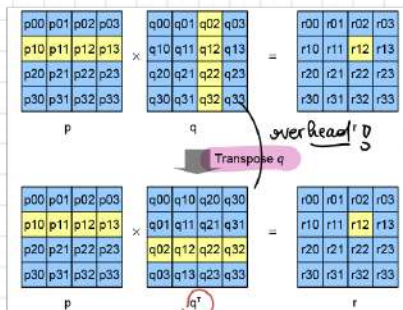
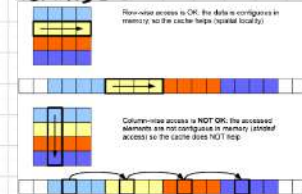
Example: matrix-matrix product nell'ottica dell'uso della cache

- Given two square matrices p, q , compute $r = p \times q$



```
void matmul(const double *p, const double *q, double *r, int n)
{
    int i, j, k;
    for (i=0; i<n; i++) {
        for (j=0; j<n; j++) {
            double s = 0.0;
            for (k=0; k<n; k++) { s += p[i*n + k] * q[k*n + j]; }
            r[i*n + j] = s;
        }
    }
}
```

REMINDER



matrice trasposta

⇒ ogni accesso per colonna è un accesso per RIGA → accesso migliore
 ⇒ scorro entrambe le matrici x riga

Ha senso?

Transposing q requires time. Is it worth it?

`make serial` will produce two executables, `matmul-plain` and `matmul-transpose`

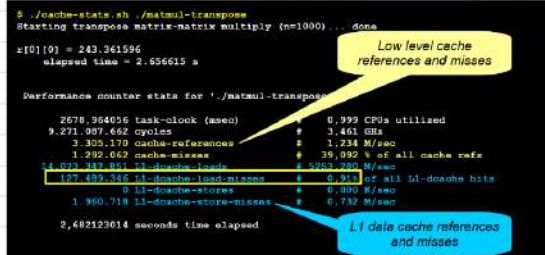
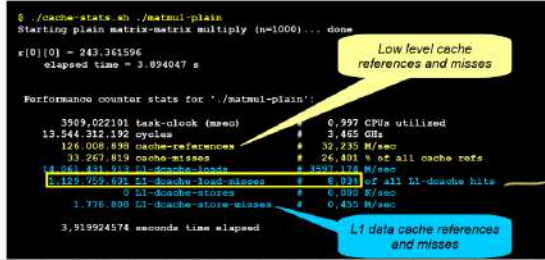
To measure the cache performance we use `perf`

- On Debian/Ubuntu:
`sudo apt-get install linux-tools-common linux-tools-generic`
- Which counters can be measured is system-dependent
- The meaning of counters is system-dependent

Nonostante ci sia la copia di una matrice in +
 ⇒ Ne vale la pena → SFRUTTO MEGLIO LE CACHE

Measuring cache stats

```
perf stat -B -e \
task-clock,cycles,cache-references,\
cache-misses,L1-dcache-loads,L1-dcache-load-misses,\
L1-dcache-stores,L1-dcache-store-misses \
./exec_name
```



Processori hanno contatori hw con overhead nullo, lo fa lui in automatico

Usarlo complesso — quali contatori? ogni processore ne ha di suoi,

numero di info, letture che il processore non ha trovato

=> migliore uso della cache anche dai dati del processore

Hardware Multi-threading

A GRANA GRASSA ~ 800 operativi dare l'illusione di avere + processi in contemporanea
operazioni context switch — costo ma giustificato
processo bloccato x 1/0

In presenza di stalli di breve durata — non efficace x costo superiore al tempo necessario
rispetto a recuperarli dalle parti

FINE! hw CPU deve supportarlo + context switch in tempo 0
no latenze ad accessi in memoria

Lo Simultaneous multithreading (x approfondire link ad articolo)
ogni core fisico visto dal SO come 2 core logici
=> esegue il doppio di processi indipendenti

all'interno di ogni core fisico ci sono risorse hw principali ALU...! condivise
tra i 2 core logici

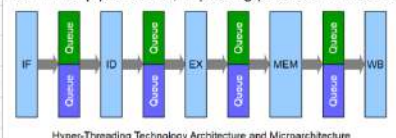
=> i due flussi aritmetico-logici deve aspettare l'altro

ma qui ha separati registri dedicati duplicati

=> 2 core logici non avranno lo stesso di 2 core fisici

x garantire e consentire ai flussi di interrompersi 1 x migliorare uso risorse hw condivise

The pipeline stages are separated by two buffers (one for each executing thread)
If one thread is stalled, the other one might go ahead and fill the pipeline slot, improving processor utilization



x eseguire istruzioni in FASI (sono + di quelle diseguate)

Instruction Fetch

Instruction Decode

Execute

Memory Access

Write Back

codice bin recuperato e capisce che tipo di istr è

se l'istr richiede access => si accede

risultati delle computazioni

Queue Queue: due flussi di istruzioni

in ogni fase solo 1 delle 2 può entrare —> l'altra aspetta

Aziende mancano fasi delle pipeline —> hw fa avanzare l'altro flusso x massimizzare l'uso di tutte le fasi

Allows the CPU to switch to another task when the current task is stalled

Fine-grained multithreading

- A context switch has essentially zero cost
- The CPU can switch to another task even on stalls of short durations (e.g., waiting for memory operations)
- Requires specific CPU support, e.g., Cray XMT

Coarse-grained multithreading

- A context switch has non-negligible cost, and is appropriate only for threads blocked on I/O operations or similar
- The CPU is less efficient in the presence of stalls of short duration

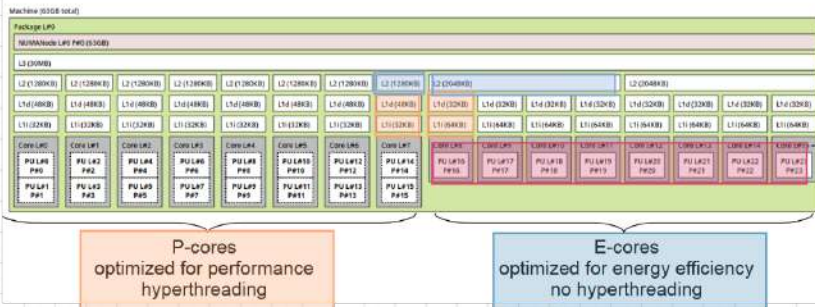
Simultaneous multithreading (SMT) is an implementation of fine-grained multithreading where different threads can use multiple different units at the same time

HyperThreading is Intel's implementation of SMT

- Each physical processor core is seen by the Operating System as two "logical" processors
- Each "logical" processor maintains a complete set of the architecture state:
 - general-purpose registers
 - control registers
 - advanced programmable interrupt controller (APIC) registers
 - some machine state registers
- Intel claims that HT provides a 15–30% speedup with respect to an equivalent, non-HT processor

Example Hyperthreading + Asymmetric cores

- Output of `lscpu` ("Thread(s) per core") or `lstopo` (*hwloc* Ubuntu/Debian package)



2 tipi di CORE diversi

- E-CORE ottimizzati x consumo energetico
- P-CORE performance

Not a Node = Not Uniform Memory Access

E-CORE condivisi su 4 // P-CORE dedicati e dimensioni diverse CACHE

CORE FISICI ognuno è una CPU indipendente
ogni logico usa 1 fisico

```
$ lscpu --all --extended
```

CPU	NODE	SOCKET	CORE	id	id1:11:12:13	ONLINE	MAXMEM	MINMEM	MEMS	freq. attuale
0	0	0	0	0:0:0:0	0	yes	5000,0000	800,0000	799,4285	
1	0	0	0	0:0:0:0	1	yes	5000,0000	800,0000	800,0000	
2	0	0	1	1:4:1:1	0	yes	5000,0000	800,0000	800,0000	
3	0	0	1	1:4:1:1	1	yes	5000,0000	800,0000	800,0000	
4	0	0	2	2:8:2:2	0	yes	5000,0000	800,0000	800,0000	
5	0	0	2	2:8:2:2	1	yes	5000,0000	800,0000	799,9740	
6	0	0	3	12:12:3:0	0	yes	5000,0000	800,0000	800,0000	
7	0	0	3	12:12:3:0	1	yes	5000,0000	800,0000	799,8160	
8	0	0	4	16:16:4:0	0	yes	5100,0000	800,0000	859,9900	
9	0	0	4	16:16:4:0	1	yes	5100,0000	800,0000	859,9870	
10	0	0	5	20:20:5:0	0	yes	5100,0000	800,0000	853,4460	
11	0	0	5	20:20:5:0	1	yes	5100,0000	800,0000	861,8560	
12	0	0	6	24:24:6:0	0	yes	5000,0000	800,0000	800,0000	
13	0	0	6	24:24:6:0	1	yes	5000,0000	800,0000	800,0000	
14	0	0	7	28:28:7:0	0	yes	5000,0000	800,0000	799,9210	
15	0	0	7	28:28:7:0	1	yes	5000,0000	800,0000	800,0000	
16	0	0	8	32:32:8:0	0	yes	3800,0000	800,0000	799,9110	
17	0	0	8	32:32:8:0	1	yes	3800,0000	800,0000	800,0000	
18	0	0	10	34:34:8:0	0	yes	3800,0000	800,0000	800,0000	
19	0	0	11	35:35:8:0	0	yes	3800,0000	800,0000	799,9380	
20	0	0	12	36:36:9:0	0	yes	3800,0000	800,0000	799,8960	
21	0	0	13	37:37:9:0	0	yes	3800,0000	800,0000	861,9310	
22	0	0	14	38:38:9:0	0	yes	3800,0000	800,0000	861,2520	
23	0	0	15	39:39:9:0	0	yes	3800,0000	800,0000	800,0000	

freq. attuale

Mostra informazioni di prima + frequenze di clock

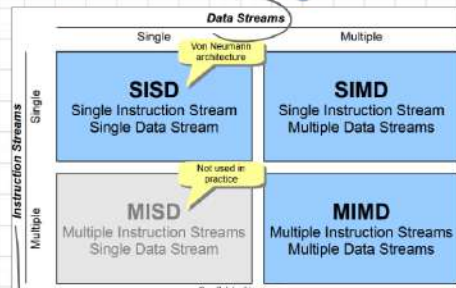
non è fisso: MIN e MAX

limitarla x meno consumo energetico

Flynn's Taxonomy

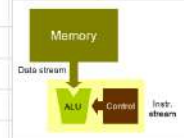
- Architetture parallele in 4 famiglie

Quanti flussi di dati



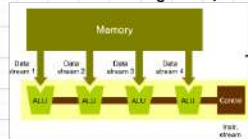
SISD

processori single-core tradizionali



SIMD

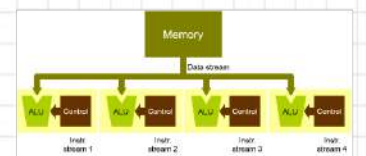
1 singolo flusso su multipli flussi di dati



→ si possono ottenere incrementi di prestazioni modesti

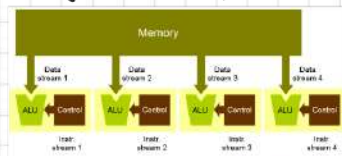
MISD

pochi esempi - difficile immaginare



MIMD

+ generali: + processi indipendenti su + flussi di dati indipendenti



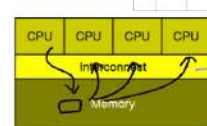
Esecuzione DAWERO CONTEMPORANEA

Shared Memory

- A set of processors sharing a common memory space
- Each processor can access any memory location

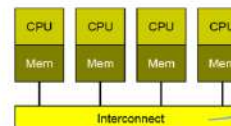
Distributed Memory

- A set of compute nodes connected through an interconnection network
- The most simple example: cluster of PCs connected via Ethernet
- Nodes can share data through explicit communications



! Race Conditions

~ Processori multi-core



~ Cluster / Super Computer

x aggiungere nodi quasi ∞

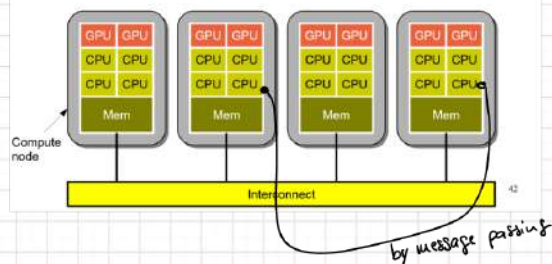
rete butterfly

x connettere in modo scalare

SuperComputer hanno architettura ibrida

- Many HPC systems are based on hybrid architectures
 - Each compute node is a shared-memory multiprocessor
 - A large number of compute nodes is connected through an interconnect network

parallelismo multicore



26/02/2026

Sum Reduction = somma del contenuto degli array

- in una architettura shared memory



ESEMPIO

ALGORITMI SEQUENZIALI scorre l'array per intero

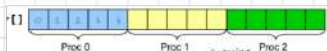
```
float seq_sum(const float* v, int n)
{
    int i;
    float sum = 0.0;
    for (i=0; i<n; i++) {
        sum += v[i];
    }
    return sum;
}
```

$\Theta(n)$ = ESATTAMENTE $O(n)$ = al massimo
↓
non può essere di-



Memoria Condivisa

Assumiamo di avere P unità di esecuzione
ciascuna della quale computa una somma parziale $\frac{n}{P}$ elementi adiacenti



```
mutex m;
sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    for (my_i = my_start; my_i < my_end; my_i++) {
        mutex_lock(&m);
        sum += v[my_i];
        mutex_unlock(&m);
    }
}
```

pragma omp atomic

modifica atomica = ogni unità di elaborazione effettua il blocco in modo indipendente dalle altre

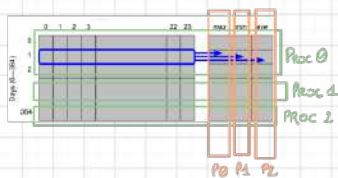
pragma omp barrier

tutte le unità di elaborazione arrivano nel punto della barrier e attendono che tutte siano arrivate lì, per poi proseguire

Task Parallelism

distribuire task sui processori

ESEMPIO Tabella con temperature orarie
calcolare MINIMA GIORNALIERA, MASSIMA e MEDIA
con 3 unità di esecuzione



partizione dei dati
a blocchi x riga

Single Program Multiple Data

usato x alg. divide-et-impera / ricorsivi

> ogni processore è specializzato in un task

Data Parallelism

Tutti i processori eseguono lo stesso task, ma su dati diversi

4 PARALLEL PATTERN

26/02/2026

Un design pattern è una soluzione generale ad un problema ricorrente

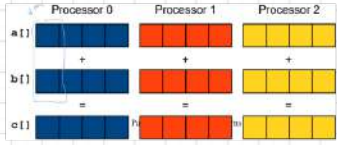
Non invento un nuovo tipo di ponte ogni volta, uso quelli già esistenti

In base al contesto e al problema da risolvere → scelgo un pattern adatto e lo modello in base alla situazione

EMBARRASSINGLY PATTERN

Quando non c'è nulla da fare

computazione divisa in task indipendenti risolvibili con poca/nessuna comunicazione tra processi



→ concatenati i risultati parziali
→ farlo già salvare nel posto giusto

Esempi di problemi

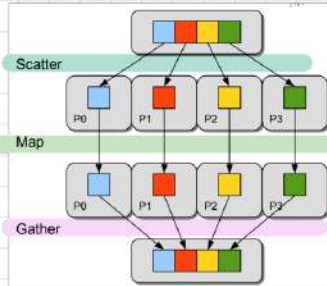
- * vector sum
- * mandelbrot set
- * 3D rendering
- * brute force password crack

SCATTER / GATHER

per problemi a memoria distribuita

struttura ad API
con blocchi da comunicare

⇒ da riassembleare



PARTITION

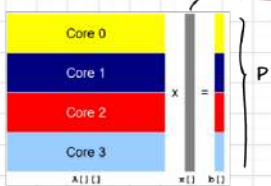
Domaino di input diviso in regioni disgiunte (= partizioni)

Come decido che forma/dimensioni hanno? → dividiamo in



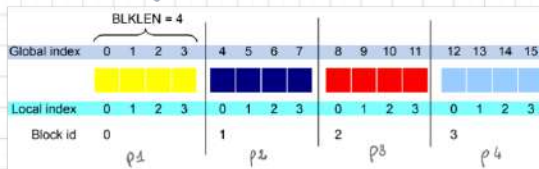
ESEMPIO Prodotto Matrice-Vettore

A → P blocchi
↓ processore



↓ Processore ha una copia di x

Block partitioning each element as a global and local index



Se tutti i blocchi hanno la stessa lunghezza

$$\Rightarrow \text{globalIndex} = \text{localIndex} + \text{blockID} \times \text{blockLength}$$

→ Associazione global - local index

$$\text{blockID} = \text{globalIndex} / \text{blockLength}$$

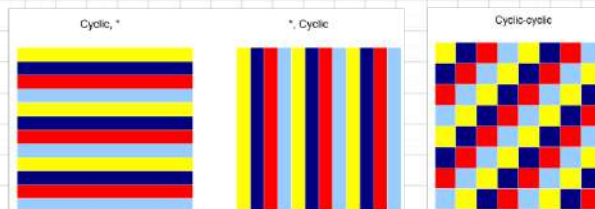
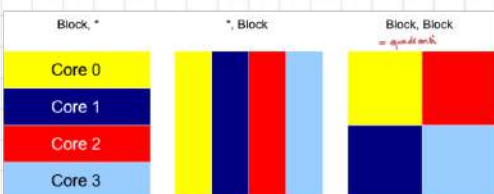
$$\text{localIndex} = \text{globalIndex} \% \text{blockLength}$$

UNIFORME o GRANA GROSSA

Cycle partitioning uniforme o grana fine



2D - con Matrici



ESEMPIO Partizione Irregolare

Superficie di un lago, può essere approssimata come una rete di triangoli

A grana fine

- ★ migliore carico di lavoro
↳ in part. con MASTER WORKER PATTERN
- ⚠ se troppo fine rischia di galleggiare
[la comunicazione sovrasta la computazione]

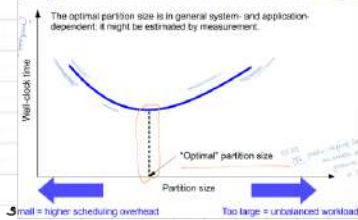


A grana grossa

- ★ migliore rapporto comunicazione/computazione
- ⚠ possibile sbilanciamento del carico

< la granularità ottimale dipende dal problema >

Scegliere la dimensione della partizione



ESEMPIO Mandelbrot

= insieme dei punti c del piano complesso | la sequenza $z_n(c)$ definita, non diverge quando $n \rightarrow +\infty$

$$z_n(c) = \begin{cases} 0 & \text{se } n=0 \\ z_{n-1}^2(c) + c & \text{altrimenti} \end{cases}$$

Successione ∞ non divergente

$$(a+ib)^2 = (a+ib)(a+ib) = a^2 + iab + iba + (ib)^2 = a^2 + 2iab + b^2$$

- Se $|z_n(c)| < 2$ dopo un massimo fissato di iterazioni
→ il pixel è colorato di nero (è insieme di Mandelbrot)
altrimenti

il colore dipende dal numero di iterazioni richieste da $|z_n(c)| > 2$

PSEUDOCODICE

$$z_n(c) = x + yi$$

molto altro

Embarrassingly parallel structure: the color of each pixel can be computed independently from other pixels

```
maxit = 1000; // il valore massimo di pixel
for each point (cx, cy) { // coordinate del bounding box
    x = y = 0; // componenti R del numero
    it = 0;
    while (it < maxit AND  $x^2 + y^2 \leq 2 \cdot 2$ ) { // x migliore ottimizzazione
        xnew = x*x - y*y + cx; // parte R del numero successivo
        ynew = 2*x*y + cy; // parte I del numero successivo
        x = xnew; // aggiornare il valore corrente
        y = ynew; // aggiornare il valore corrente
        it = it + 1; // aggiornare il contatore delle iterazioni
    }
    plot(cx, cy, it); // colore una parte
}
```

sizeof(.) : quanti byte occupa

diventa falso quando $(it \geq maxit) \vee (x^2 + y^2 \geq 2^2)$

Per parallelizzarlo, posso provare a partizionarlo a grana grossa x righe

↳ ma il lavoro non sarebbe distribuito in modo equo → **SBILANCIAMENTO DEL CARICO**

i pixel di Mandelbrot impiegano meno iterazioni

→ Idealmente deve essere un sistema di **LOAD BALANCING** dei processori

```

91
92 //
93 * Iterate the recurrence:
94 *
95 * z_0 = 0;
96 * z_{n+1} = z_n^2 + (cx + i*cy);
97 *
98 * Returns the first n such that ||z_n|| > 2, or [MAXIT]
99 */
100 int iterate( float cx, float cy )
101 {
102     float x = 0.0f, y = 0.0f, xnew, ynew;
103     int it;
104     for ( it = 0; (it < MAXIT) && (x*x + y*y <= 2.0*2.0); it++ ) {
105         xnew = x*x - y*y + cx;
106         ynew = 2.0*x*y + cy;
107         x = xnew;
108         y = ynew;
109     }
110     return it;
111 }

```

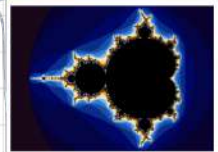
parallelizzare i pixel

```

175 const double tstart = omp_get_wtime();
176 #pragma omp parallel for schedule(runtime) default(none) shared(xsize,ysize,XMIN,XMAX,YMIN,YMAX)
177 for ( int y = 0; y < ysize; y++ ) {
178     for ( int x = 0; x < xsize; x++ ) {
179         const double re = XMIN + (XMAX - XMIN) * (float)(x) / (xsize - 1);
180         const double im = YMAX - (YMAX - YMIN) * (float)(y) / (ysize - 1);
181         const int nit = iterate(re, im);
182         drawpixel( x, y, nit );
183     }
184 }
185 const double elapsed = omp_get_wtime() - tstart;
186 printf("Execution time %.3f\n", elapsed);

```

(0,0)



```

113 /*
114 * Draw a pixel at window coordinates (x, y) with the appropriate
115 * color, depending on the number of iterations it; (0,0) is the upper
116 * left corner of the window, y grows downward.
117 */
118 void drawpixel( int x, int y, int it )
119 {
120     #ifdef GUI
121     #pragma omp critical
122     {
123         if (it < MAXIT) {
124             gfx_color( COLORS[it % NCOLORS].r,
125                       COLORS[it % NCOLORS].g,
126                       COLORS[it % NCOLORS].b );
127         } else {
128             gfx_color( 0, 0, 0 );
129         }
130         gfx_point( x, y );
131     }
132     #else
133     pixel_t *p = &bitmap[y*xsize + x];
134     if (it < MAXIT) {
135         p->r = COLORS[it % NCOLORS].r;
136         p->g = COLORS[it % NCOLORS].g;
137         p->b = COLORS[it % NCOLORS].b;
138     } else {
139         p->r = p->g = p->b = 0;
140     }
141     #endif
142 }

```

sotto componenti RGB

LOAD BALANCING (= il carico di lavoro è bilanciato su ogni processo per far più o meno la stessa quantità di lavoro)

1 usare **PARTIZIONAMENTO** a grana fine
utile quando si hanno più task, che processori
! attenzione alle possibili comunicazioni tra task

2 usare **ALLOCAZIONE DINAMICA (MASTER-WORKER)**

ho dei task da assegnare ed eseguire, ma non so la loro durata fino a quando non li eseguo



ne assegno uno ad ogni processore → quando il task termina, il processore viene riallocato con il successivo task in coda (il 1° disponibile dal worker)

- non uso un'assegnazione dinamica dei task
- riduco la granularità dei task

STENCIL = pattern geometrico fisso con Dominio = {array | matrici}



- ha bisogno di una COMPUTAZIONE SINCRONA

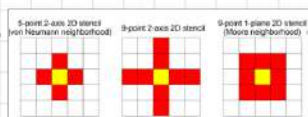
↳ un'unità di esecuzione x cella

accede ai vicini allo stesso tempo → computa il nuovo valore e lo salva nel registro locale NEXT Domain

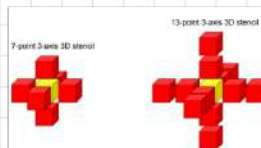
Si aspetta che tutti abbiano finito

Tutte le unità scrivono il nuovo valore nel dominio

STENCIL 2D



STENCIL 3D



#1

! le architetture parallele non sono sincrone → possiamo simulare usando 2 domini

#2

! gestione delle celle sui bordi → A in alcuni problemi, hanno valori fissati

→ B in altri, Dominio ciclico → superficie toroidale

→ C Estendere il Dominio con GHOST CELLS — le celle sul bordo fanno da effetto specchio

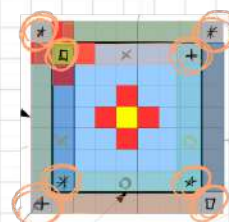
↓
< Come si riempiono le GHOST CELLS ? >

A o colonna della matrice
prendo una riga e copiarla nella ghost area dal lato opposto

[! elementi in angolo servono con stencil quadrati]

! copiare dati → overhead → ne vale la pena? Sì
↳ prop alla quantità di dati sul bordo ~ $\Theta(n)$

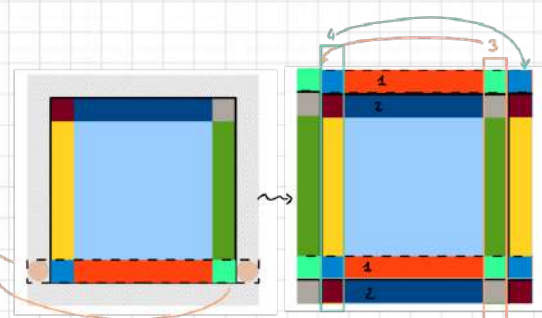
↳ incremento delle prestazioni



B inizializzazione della GHOST AREA (inclusi gli angoli)

• prendo la 1° riga/colonna del Dominio e le 2 aree fuori nella GHOST AREA e la copio nel lato opposto, all'interno della Ghost Area

• dopo aver copiato le prime 2, copiando 3, 4 avrò anche gli angoli della GHOST AREA



ESEMPIO Game of life

Dominio 2D : ogni cella può essere $\begin{cases} \text{VIVA} = 1 \\ \text{MORTA} = 0 \end{cases}$

lo stato della cella al tempo $t+1$, dipende $\begin{cases} \text{dallo stato della cella al tempo } t \\ \text{num di celle vive al tempo } t \text{ tra gli 8 vicini} \end{cases}$

REGOLE

- una cella VIVA con meno di 2 vicini VIVI $\rightarrow$ MUORE
- una cella VIVA con 2/3 vicini VIVI $\rightarrow$ VIVE
- una cella VIVA con più di 3 vicini VIVI $\rightarrow$ MUORE
- una cella MORTA con 3 vicini VIVI $\rightarrow$ VIVE

```
40 #include <stdio.h>
41 #include <stdlib.h>
42
43 /* grid size (excluding ghost cells) */
44 #define SIZE 256
45
46 int cur = 0; /* index of current grid (must be 0 or 1) */
47 unsigned char grid[2][SIZE+2][SIZE+2];
48
49 /* some useful constants; starting and ending rows/columns of the domain */
50 const int LEFT = 1;
51 const int RIGHT = SIZE;
52 const int TOP = 1;
53 const int BOTTOM = SIZE;
```

quali delle 2 è il Dominio? $\rightarrow$ l'ultima colonna ha index SIZE+1

stencil applicato tutto intorno al Dominio vero

STENCIL	GHOST AREA
3x3	1
5x5	2
7x7	3

```
75 /* copy the sides of current grid to the ghost cells. This function
76 uses the global variables cur and grid. grid[cur] is modified. */
77 #if 1
78 void copy_sides( void )
79 {
80     const int HALO_TOP = TOP-1;
81     const int HALO_BOTTOM = BOTTOM+1;
82     const int HALO_LEFT = LEFT-1;
83     const int HALO_RIGHT = RIGHT+1;
84
85     /* copy top and bottom (one should better use memcpy()) */
86     for (int j=LEFT; j<=RIGHT; j++) {
87         grid[cur][HALO_TOP][j] = grid[cur][BOTTOM][j];
88         grid[cur][HALO_BOTTOM][j] = grid[cur][TOP][j];
89     }
90     /* copy left and right */
91     for (int i=TOP; i<=BOTTOM; i++) {
92         grid[cur][i][HALO_LEFT] = grid[cur][i][RIGHT];
93         grid[cur][i][HALO_RIGHT] = grid[cur][i][LEFT];
94     }
95     /* copy corners */
96     grid[cur][HALO_TOP][HALO_LEFT] = grid[cur][BOTTOM][RIGHT];
97     grid[cur][HALO_TOP][HALO_RIGHT] = grid[cur][BOTTOM][LEFT];
98     grid[cur][HALO_BOTTOM][HALO_LEFT] = grid[cur][TOP][RIGHT];
99     grid[cur][HALO_BOTTOM][HALO_RIGHT] = grid[cur][TOP][LEFT];
100 }
101 #else
102 /* Another way to fill the ghost cells: change the ranges of the "for"
103 cycles to copy entire rows and columns (including ghost cells). At
104 the end, corners get filled with the correct values (draw an
105 example to convince yourself). The interesting thing is that you
106 can swap the two "for" cycles (i.e., first handle columns, then
107 handle rows) and the final result is still correct. */
```

```
void copy_sides( )
{
    const int HALO_TOP = TOP-1;
    const int HALO_BOTTOM = BOTTOM+1;
    const int HALO_LEFT = LEFT-1;
    const int HALO_RIGHT = RIGHT+1;

    /* Copy top and bottom (one can also use memcpy()). We copy a
    whole row (including ghost cells). */
    for (int j=0; j<=HALO_RIGHT; j++) {
        grid[cur][HALO_TOP][j] = grid[cur][BOTTOM][j];
        grid[cur][HALO_BOTTOM][j] = grid[cur][TOP][j];
    }

    /* Copy left and right. We copy a whole column (including ghost
    cells). */
    for (int i=0; i<=HALO_BOTTOM; i++) {
        grid[cur][i][HALO_LEFT] = grid[cur][i][RIGHT];
        grid[cur][i][HALO_RIGHT] = grid[cur][i][LEFT];
    }

    /* There is no need to fill the corners */
}
```

```
133 /* Compute the next grid given the current configuration; this
134 function uses the global variables grid and cur; updates are
135 written to the (1-cur) grid. */
136 void step( void )
137 {
138     int next = 1 - cur;
139     for (int i=TOP; i<=BOTTOM; i++) {
140         for (int j=LEFT; j<=RIGHT; j++) {
141             /* count live neighbors of cell (i,j) */
142             const int nbors =
143                 grid[cur][i-1][j-1] + grid[cur][i-1][j] + grid[cur][i-1][j+1] +
144                 grid[cur][i][j-1] + grid[cur][i][j] + grid[cur][i][j+1] +
145                 grid[cur][i+1][j-1] + grid[cur][i+1][j] + grid[cur][i+1][j+1];
146             /* apply rules of the game of life to cell (i,j) */
147             if ( grid[cur][i][j] && (nbors < 2 || nbors > 3) ) {
148                 grid[next][i][j] = 0;
149             } else {
150                 if ( !grid[cur][i][j] && (nbors == 3) ) {
151                     grid[next][i][j] = 1;
152                 } else {
153                     grid[next][i][j] = grid[cur][i][j];
154                 }
155             }
156         }
157     }
158     /* swap sul valore i j
```

calcolo il numero di vicini VIVI

REDUCE = applicazione di un operatore binario associativo [: somma | prodotto | min | max | ...] agli elementi di un array

↳ può essere realizzato in $O(\log n)$ Step paralleli

sum-reduce → parsing in parallel

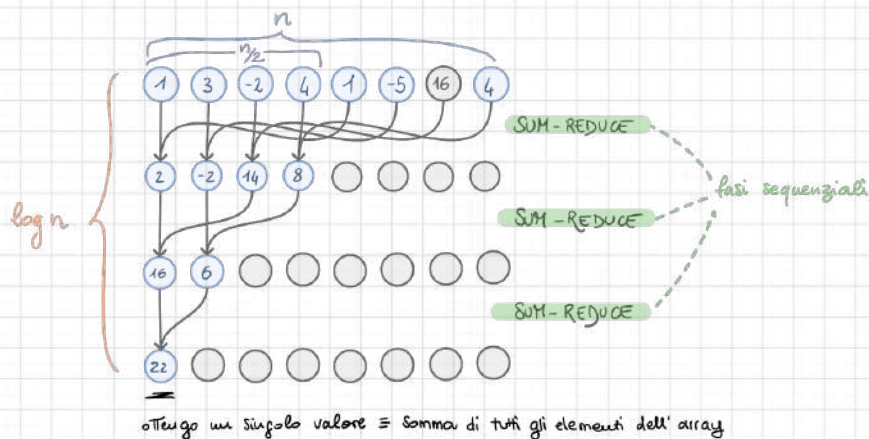
↳ valore della 1^a metà + ↳ valore della 2^a metà

⇒ ottengo un array più piccolo

```
int n2 = n, i;
do {
    n2 = (n + 1) / 2;
    for (i = 0; i < n2; i++) {
        if (i + n2 < n) x[i] += x[i + n2];
    }
    n = n2;
} while (n2 > 1);
return x[0];
```

Can be done in parallel!

See reduction.c



```
34 /* Serial implementation of the sum-reduce operator */
35 float sum_reduce_serial( float* x, int n )
36 {
37     float result = 0.0;
38     for (int i=0; i<n; i++) {
39         result += x[i];
40     }
41     return result;
42 }
43
44 /* Tree-structured implementation of the sum-reduce operator; this
45    implementation works for any vector length n > 0 */
46 float sum_reduce( float *x, int n )
47 {
48     int n2;
49     do {
50         n2 = (n+1)/2; // n/2 rounded up ← Sfrutto la divisione dell'array n/2
51         for (int i=0; i<n2; i++) {
52             if (i+n2<n) x[i] += x[i+n2];
53         }
54         n = n2;
55     } while (n2 > 1);
56     return x[0];
57 }
```

SCAN = computa tutti i prefissi di un array usando un operatore binario associativo

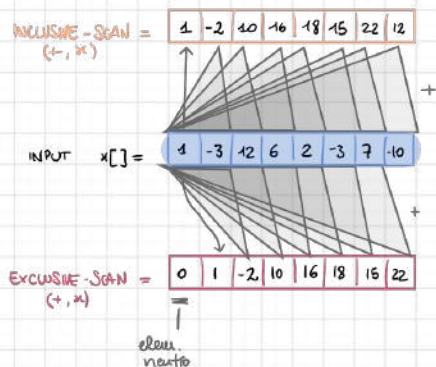
array $[y_0, y_1, \dots, y_{n-1}] = \text{INCLUSIVE-SCAN}(\text{operatore}, \text{array}[x_0, x_1, \dots, x_{n-1}])$ dove

$y_0 = x_0$
 $y_1 = x_0 \text{ operatore } x_1$
 $y_2 = x_0 \text{ operatore } x_1 \text{ operatore } x_2$
 $y_{n-1} = x_0 \text{ operatore } x_1 \text{ operatore } \dots \text{ operatore } x_{n-1}$

array $[y_0, y_1, \dots, y_{n-1}] = \text{EXCLUSIVE-SCAN}(\text{operatore}, \text{array}[x_0, x_1, \dots, x_{n-1}])$ dove

$y_0 = 0$ ← elem. minimo dell'elemento neutro $1 \oplus x = x \oplus 1 = x$
 $y_1 = x_0$
 $y_2 = x_0 \text{ operatore } x_1$
 $y_{n-1} = x_0 \text{ operatore } x_1 \text{ operatore } \dots \text{ operatore } x_{n-2}$
 $\min(I, x) = x$
 $\rightarrow I = +\infty$

ESEMPIO Scan con SOMMA



IMPLEMENTAZIONE SERIALE

```
void inclusive_scan( int *x, int *s, int n ) {
    int i;
    s[0] = x[0];
    for (i = 1; i < n; i++) {
        s[i] = s[i-1] + x[i];
    }
}

void exclusive_scan( int *x, int *s, int n ) {
    int i;
    s[0] = 0;
    for (i = 1; i < n; i++) {
        s[i] = s[i-1] + x[i-1];
    }
}
```

! alg $O(n^2)$ → ma abbiamo $O(n)$ per calcolo del precedente

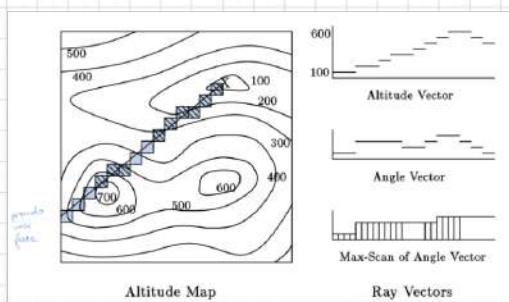
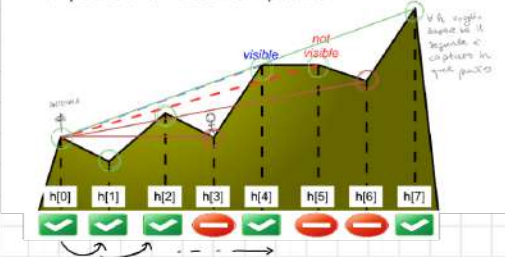
? Come si parallelizza? dove leggere $s[i-1]$

LOOP CARRIED DEPENDENCE!

ESEMPLO Line of sight

n peaks of heights $h[0], \dots, h[n-1]$; the distance between consecutive peaks is one.

Which peaks are visible from peak 0?



Se dovessimo risolverlo in modo sequenziale $O(n)$

guardiamo un picco dopo l'altro e vediamo se è visibile o meno rispetto al 0

calcolo l'angolo massimo tra $h[0]$ e il corrente picco confrontandolo con tutti i picchi precedenti

Se $>$ è raggiunto dal segnale posso aggiornare la variabile del picco massimo

```

for each i = 0, ... n-1
  - Let  $a[i]$  be the slope of the line connecting the peak 0 to the peak i
  -  $a[0] = 0$ 
  -  $a[i] = \arctan((h[i] - h[0]) / i)$ , se  $i > 0$ 
for each i = 0, ... n-1
  -  $amax[0] = 0$ 
  -  $amax[i] = \max(a[0], a[1], \dots, a[i-1])$ , se  $i > 0$ 
for each i = 0, ... n-1
  - If  $a[i] \geq amax[i]$  then the peak i is visible
  - otherwise the peak i is not visible
    
```

```

bool[0..n-1] Line-of-sight( double h[0..n-1] )
bool v[0..n-1]
double a[0..n-1], amax[0..n-1]
a[0] = 0
for i = 1 to n-1 do
  a[i] = arctan( ( h[i] - h[0] ) / i )
endfor
amax[0] = 0
for i = 1 to n-1 do
  amax[i] = max( a[i-1], amax[i-1] )
endfor
for i = 0 to n-1 do
  v[i] = ( a[i] >= amax[i] )
endfor
return v
    
```

SERIALE

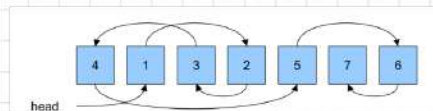
```

bool[0..n-1] Parallel-line-of-sight( double h[0..n-1] )
bool v[0..n-1]
double a[0..n-1], amax[0..n-1]
a[0] = 0
for i = 1 to n-1 do in parallel
  a[i] = arctan( ( h[i] - h[0] ) / i )
endfor
amax = exclusive-scan( max, a )
for i = 0 to n-1 do in parallel
  v[i] = ( a[i] >= amax[i] )
endfor
return v
    
```

PARALLELO

ESERCIZIO List Ranking - Pointer Jumping

ETICHETTARE TUTTI I NODI DI UNA LISTA



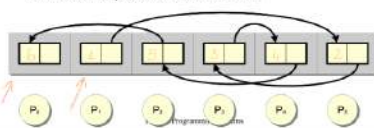
```

typedef struct list_node {
  int rank;
  struct list_node *next;
} list_node;

void rank( list_node *item )
{
  int r = 0;
  while ( item != NULL ) {
    item->rank = r++;
    item = item->next;
  }
}
    
```

Data representation

- We use n processors to rank a list of n nodes.
- Each processor is associated to a node.
- To make the association easier, the list nodes are stored in an array.



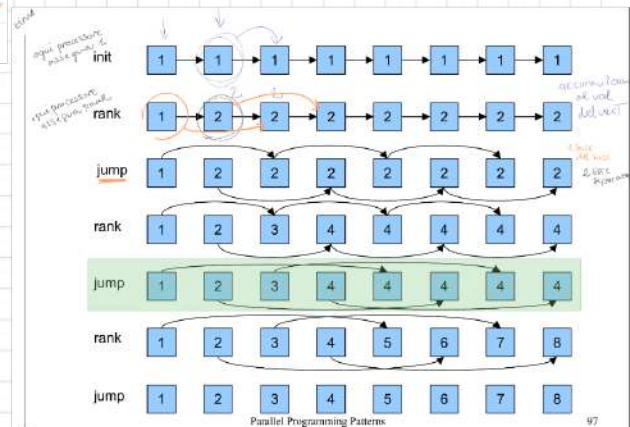
Pointer Jumping

- One processor per list node.
- Init:
 - Each processor sets $item \rightarrow rank = 1$
- Main loop to be executed **synchronously** by all processors
 - $if (item \rightarrow next != NULL)$
 - $item \rightarrow next \rightarrow rank = item \rightarrow rank + 1$ (rank)
 - All processors read $item \rightarrow next \rightarrow rank$ at the same time, and store the value in a local register.
 - All processors read $item \rightarrow rank$ at the same time, and store the value in a local register.
 - All processors update $item \rightarrow next \rightarrow rank$ at the same time.
 - $if (item \rightarrow next != NULL)$
 - $item \rightarrow next = item \rightarrow next \rightarrow next$ (jump)
 - All processors read $item \rightarrow next \rightarrow next$ at the same time, and store the value in some local register.
 - All processors set $item \rightarrow next$ to that value at the same time.

Span

- If the list has n nodes, the main loop requires $O(\log n)$ iterations.
 - The first *jump* splits the list into two sublists.
 - Subsequent *jump* split the lists into two halves.
 - After at most $O(\log n)$ *jump*, lists have a single element.
- Each iteration can be executed in parallel by $O(n)$ processors
 - Even if we relax the synchronous execution requirement.
 - We will see in the lab...

- A parallel programming patterns defines:
 - a partitioning of the input data
 - a communication structure among parallel tasks
- Parallel programming patterns can help to define efficient algorithms
 - Many problems can be solved using one or more known patterns



5 Memoria Condivisa su OpenMP

OpenMP modello di programmazione parallela a memoria condivisa, portabile su diversi tipi di architetture

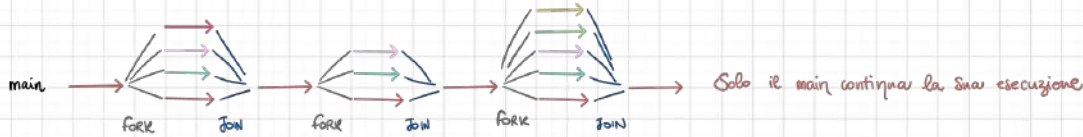
- * a partire da un programma seriale → lo si parallelizza in alcune componenti
- * si basa sul compilatore, che genera threads e sincronizzazione con costrutti
- * è un'estensione di linguaggi esistenti [Fortran, C, C++]

Non parallelizza automaticamente

Non garantisce lo speedup — la gestione dei thread non ha costo 0

Non impedisce Data Races

EXECUTION MODEL Fork-Join



al momento della Join, si aspetta prima che tutte le unità di elaborazione abbiano finito → `#pragma omp barrier` implicita

direttive pragma

sono speciali costrutti del pre-processore che possono essere a blocchi {} o inline [come in C]

`#pragma omp parallel` crea una regione parallela in cui si crea un team di thread gestiti dal main (con ID 0)



ha dimensioni che dipende dall'implementazione

↳ V thread si creerà una copia del codice della regione che verrà eseguita da ogni rispettivo thread

! alla fine della regione c'è una **barriera implicita** dopo la quale solo il main procederà con l'esecuzione

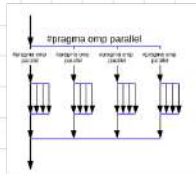
```
#pragma omp parallel [clause ...]
clause ::=
if (scalar_expression) |
private (list) |
shared (list) |
default (shared | none) |
firstprivate (list) |
reduction (operator: list) |
copyin (list) |
num_threads(thr)
```

`omp_get_num_threads()` → ritorna il numero di thread nel team attivo in questo momento || 1

`omp_get_max_threads()` → ritorna il numero di thread massimo che possono essere creati

`omp_get_thread_num()` → ritorna l'ID del thread chiamante (che è al team più interno)

posso abilitare Team Inheritance con `OMP_NESTED = true` ma è raramente utile, può causare **oversubscription** (se ci fossero + thread che processori)



`omp_get_wtime()` → ritorna l'orario attuale
↳ utile per calcolare il tempo di esecuzione

SCOPE delle VARIABILI

`shared(x)` tutti i thread accedono alla stessa locazione di memoria

`private(x)` ogni thread ha la sua copia privata
tutte le istanze non sono inizializzate
vive solo nella regione parallela

`firstprivate(x)` ogni thread ha la sua copia privata
tutte le istanze sono inizializzate col valore corrente
vive solo nella regione parallela

`default(shared)` tutte le variabili che non sono specificate, avranno questo scope
`default(none)` ← devi essere tu a specificare lo scope per tutte le variabili

ESERCIZIO Qual è l'output?

```
/* omp-scope.c */
#include <stdio.h>

int main( void )
{
    int a=1, b=1, c=1, d=1;
    #pragma omp parallel num_threads(10) \
    private(a) shared(b) firstprivate(c)
    {
        printf("Hello World!\n");
        a++;
        b++;
        c++;
        d++;
    }
    printf("a=%d\n", a);
    printf("b=%d\n", b);
    printf("c=%d\n", c);
    printf("d=%d\n", d);
    return 0;
}
```

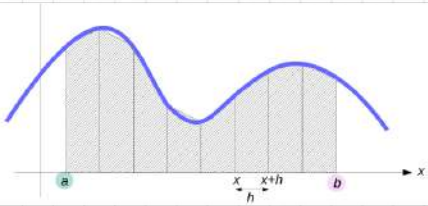
Hint: compile with -Wall

Th 0
 P a=1
 S b=2
 F c=1
 d=1

Th 1
 > Hw
 a=1
 c=2
 d=1

...
 L
 Hw (x10)
 a=1
 b=1
 c=1
 d=1

La Regola del Trapezio



$h = (b-a)/n$
 result = 0
 $x = a$
 for (int i=0; i<n; i++) {
 result += h * (f(x) + f(x+h)) / 2
 x += h
 }
 return result

possiamo assegnare un trapezio ad ogni thread

#pragma omp atomic

protegge gli aggiornamenti su una variabile shared
 LEASI → ACQUIRIZIONE → SCRIVI
 ? non protegge dai race conditions sull'accesso a var shared fuori dal blocco

#pragma omp critical

protegge l'accesso ad aree critiche: tutti i thread ci possono accedere, ma solo uno alla volta può essere nell'area critica
 ? più pesante di atomic perché richiede l'acquisizione di un lock → causa overhead + riduce la scalabilità
 ? non protegge dai race conditions sull'accesso a var shared fuori dal blocco

#pragma omp parallel reduction(OP : VAR)

è una computazione che applica ripetutamente lo stesso operatore OP ad una sequenza di operandi x risultato
 migliore alternativa di atomic
 ES: $0 \text{--} \text{reduce}(x_0, x_1, \dots, x_n) = x_0 \text{ OP } x_1 \text{ OP } \dots \text{ OP } x_n$

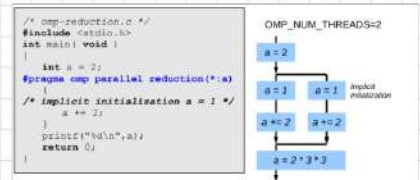
Operator	Initial value
+	0
*	1
-	0
/	1
min	largest positive number
max	most negative number
&	0
	0
^	0
&&	0
	1

Come funziona?

- ogni thread ha una copia privata della VAR di riduzione

inizializzata all'elemento neutro dell'OP di riduzione

- ogni thread esegue il blocco in parallelo
- quando tutti hanno finito → l'OP di riduzione viene applicato all'ultimo valore di ogni copia privata della VAR
 E
 al valore della VAR prima del blocco parallelo



#pragma omp for

si usa all'interno di un blocco parallelo
 assegna ogni iterazione di un ciclo ad un thread e team del blocco
 ? la variabile indice su cui iteriamo è dichiarata dalla direttiva
 + può essere unita alla direttiva #pragma omp parallel for

```
for (index = start; index < end; index++) {
    // body
}
```

devono avere type compatibili e non possono cambiare durante l'esecuzione

modificabile solo dall'incremento del ciclo

? Non si può usare se sono presenti data dependance → causerebbe LOOP CARRIES DEPENDENCY

```
double factor = 1.0;
double sum = 0.0;
for (int k=0; k<n; k++) {
    sum += factor / (2*k + 1);
    factor = -factor;
}
pi_approx = 4.0 * sum;
```

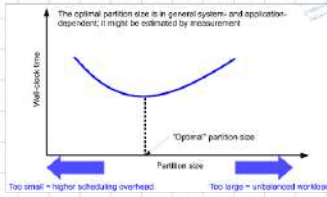
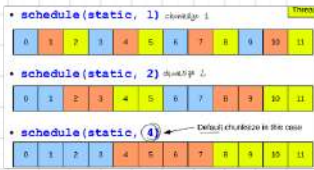
NO! (due frecce circolari tra le due linee di codice)

```
double factor;
double sum = 0.0;
#pragma omp parallel for private(factor) reduction(+:sum)
for (int k=0; k<n; k++) {
    if (k % 2 == 0) {
        factor = 1.0;
    } else {
        factor = -1.0;
    }
    sum += factor / (2*k + 1);
}
pi_approx = 4.0 * sum;
```

Factor must have private scope

schedule (TYPE, CHUNKSIZE)

- posso decidere la granularità delle porzioni gestite dai thread
- $L = \lceil N \text{ iteraz} / n \text{ thread} \rceil \times \text{decidere quali iterazioni sono assegnate a quali threads}$
- TYPE — statico iterazioni raggruppate in dimensioni CHUNKSIZE, assegnate ciclicamente ai threads
- * carico predeterminato e prevedibile → logica a compile-time
 - dinamico iterazioni assegnate ai threads tramite MASTER WORKER → il 1° che arriva prende tutto
 - * carico non prevedibile e con molte variabili → logica a runtime
 - auto assegnamento gestito dal compilatore / runtime system
 - runtime assegnamento determinato a runtime con OMP_SCHEDULE = "TYPE, CHUNKSIZE" ← tipo schedule che assegnerai al pragma omp for



#pragma omp parallel for collapse(N)

specifica quanti cicli in un ciclo annidato devono essere compressi in un unico grande spazio di iterazione (diviso in base alla direttiva schedule)

ESEMPIO di come funziona

```
#pragma omp parallel for num_threads(5) collapse(2)
for (int i=0; i<12; i++) {
    for (int j=0; j<5; j++) {
        do_work(i,j);
    }
}
```

i \ j	0	1	2	3	4
0	0,0	0,1	0,2	0,3	0,4
1	1,0	1,1	1,2	1,3	1,4
2	2,0	2,1	2,2	2,3	2,4
3	3,0	3,1	3,2	3,3	3,4

Thread 0
Thread 1
Thread 2
Thread 3
Thread 4

possiamo scrivere le iterazioni come array per assegnarle ai thread

0,0 0,1 0,2 0,3 0,4 1,0 1,1 1,2 1,3 1,4 2,0 2,1 2,2 2,3 2,4 3,0 3,1 3,2 3,3 3,4

; assegnamento di default è IMPLEMENTATION DEPENDANT

Mapping Threads to CPUs

La maggior parte dei processori multi-thread sono omogenei nella distribuzione del carico di lavoro, sfruttando la loro posizione fisica e virtuale

OMP_PLACES = "numList"

var d'ambiente usata quando non si ha un processore omogeneo, per specificare dove devono essere posizionati i threads

"0,1,2,3" || "0:4" = {0,1,2,3} associazione thread-CPU

ESEMPIO

OMP_PROC_BIND = true

previene che lo scheduler del OS migri i thread ad una diversa CPU

* permette di aggiornare meglio la cache nel thread originale

! impedisce al OS di allocare

OMP_DISPLAY_ENV = true

mostra prima della compilazione, quali env sono impostate con i rispettivi valori

È utile vale la pena assegnare i threads? In certi casi no: compilatori | os | processori diversi

#pragma omp barrier

tutti i thread devono raggiungere questo punto per proseguire

#pragma omp master

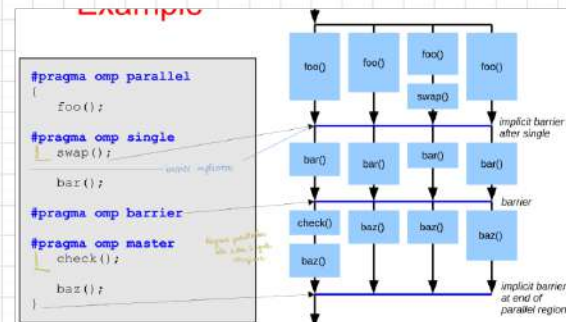
questo blocco che sto definendo è eseguito solo dal thread ID=0

! gli altri lo saltano e proseguono senza interruzioni

#pragma omp single

questo blocco è eseguito dal 1° thread che arriva

! c'è una barrier implicita alla fine del blocco



OpenMP tasks

sono unità di lavoro indipendenti all'interno della stessa area parallela

- ↳ codice da eseguire
- ↳ dati da processare con

• possono essere ANNIDATI → generare altri

I thread sono ASSEGNATI per eseguire il lavoro dei tasks

IMMEDIATAMENTE

IN UN ALTRO MOMENTO

ESEMPIO

1 thread → creo questi task

```
#pragma omp parallel
{
    #pragma omp task
    fred();
    #pragma omp task
    vilma();
    #pragma omp task
    barney();
}
```

4 thread → divido i task

```
#pragma omp parallel
{
    #pragma omp single
    {
        #pragma omp task
        fred();
    }
    #pragma omp task
    vilma();
    #pragma omp task
    barney();
}
```

task e variabili

vengono valutate al momento di esecuzione di un task (≠ creazione del task)

• posso specificare lo scoping nella direttiva `#pragma omp task`

shared → si riferiscono alla MEMORIA con quel nome quando è stato creato il task

private → si riferiscono alla NUOVA MEMORIA non inizializzata, creata quando viene eseguito il task

firstprivate → si riferiscono alla NUOVA MEMORIA creata e inizializzata con valore esistente al momento di creazione del task

+ comune meglio eseguire il task dopo = evitare problemi di scoping

ESEMPIO con linked list

```
int a = 1;
void foo( void )
{
    int b = 2, c = 3;
    #pragma omp parallel private(b)
    {
        int d = 4; // a, c shared; b private, d private
        #pragma omp task
        {
            int e = 5;
            // Scope of a: shared, value 1
            // Scope of b: firstprivate, uninitialized
            // Scope of c: shared, value 3
            // Scope of d: private => firstprivate, value 4
            // Scope of e: private, value 5
        }
    }
}
```

a=1 b=? c=3 d=4 e=5

↳ è privata con valore iniziale indefinito
di privata con valore iniziale = 4

→ task creati assegnati a thread

`#pragma omp taskwait`

aspetta fino a quando tutti i task definiti nel task corrente sono completati

ESEMPIO Fibonacci

```
#include <stdio.h>
int fib( int n )
{
    int n1, n2;
    if ( n < 2 )
        return 1;
    else
    {
        n1 = fib(n-1);
        n2 = fib(n-2);
        return n1 + n2;
    }
}

int main( int argc, char* argv[] )
{
    int n = 10, res;
    res = fib(n);
    printf("fib(%d)=%d\n", n, res);
    return 0;
}
```

$$f_n = f_{n-1} + f_{n-2}$$

$$- f_0 = f_1 = 1$$

$O(2^n)$

→

```
#include <stdio.h>
int fib( int n )
{
    int n1, n2;
    if ( n < 2 )
        return 1;
    else
    {
        #pragma omp task shared(n1)
        n1 = fib(n-1);
        #pragma omp task shared(n2)
        n2 = fib(n-2);
        #pragma omp taskwait
        return n1 + n2;
    }
}

int main( int argc, char* argv[] )
{
    int n = 10, res;
    #pragma omp parallel
    #pragma omp master
    {
        res = fib(n);
        printf("fib(%d)=%d\n", n, res);
    }
    return 0;
}
```

- Binary tree of tasks
- A task cannot complete until all tasks below it in the tree are complete
 - enforced with `taskwait`
- `n1, n2` are private to the current task
 - because they are local to a function called in a parallel block
 - So they would be `firstprivate` by default
- Must be `shared` on child tasks because their value must be stored at their parent

MP Programming

106

Riassumendo

- OpenMP è uno standard della programmazione parallela su sistemi a memoria condivisa — con `#pragma` [funzioni speciali e direttive]
- I programmi OpenMP si avviano su thread multipli → default — PARTIZIONAMENTO a BLOCCHI
- SCOPE di una variabile è la collezione di threads ai quali la var è accessibile
- REDUCE è una computazione che applica più volte l'OPERATORE ad una sequenza di operandi → per ottenere un unico risultato

6 Parallelizzare Cicli

= trasformare i cicli preservandone la semantica

In alcuni casi, le iterazioni sono indipendenti → possono essere eseguite in parallelo

↳ il num di unità di esecuzione può essere molto inferiore al numero di iterazioni ⇒ si occupa di + iterazioni senza un ordine preciso

Non tutti possono essere parallelizzati x gestione della memoria

Data Dependence = 2 accessi alla stessa locazione di memoria [di cui una in scrittura]
 ↳ Impone l'ordine di esecuzione di alcune istruzioni

RAW - Read After Write S1 → S2 Data Flow o True Dependence
 S1: a = b + c
 S2: d = 2 * a

WAR - Write After Read S1 → S2 Antidependence
 S1: c = a + b
 S2: a = 2 * b

WAW - Write After Write S1 → S2 Output Dependence
 S1: a = k
 if (a > 0)
 S2: a = 2 * c

Thm fondamentale della Dipendenza

Qualsiasi trasformazione di riordinamento che preservi ogni dipendenza in un programma, preserva il significato di quel programma

OSSERVAZIONE Riconoscere i cicli paralleli:

- trovare le Data Dependence in un ciclo
- Nessuna dipendenza che attraversa il limite di iterazione, può essere parallelizzata

ESEMPI

✓ ogni iterazione legge/scrive in locazioni diverse della memoria
 ↳ possono essere eseguiti in un ordine qualsiasi

```
for (i=0; i<n; i++) {
    S1 a[i] = b[i] + c[i];
}
```

✗ a[i] è letto e scritto RAW
 ⇒ LOOP CARRIED DEPENDENCE

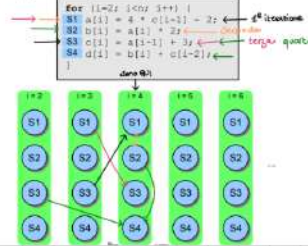
```
for (i=1; i<n; i++) {
    S1 a[i] = a[i-1] + b[i];
}
```

Deve essere implementato diversamente per parallelizzarlo → HA UNA TRASFORMAZIONE TRIVIALE

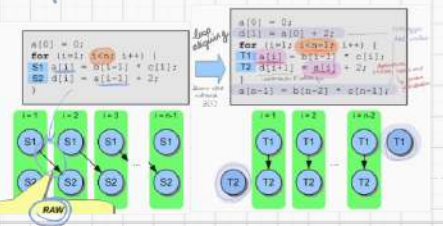
✓ (somma di un array)
 LOOP CARRIED DEPENDENCE
 può essere rimossa da una Trasformazione non triviale la REDUCE (+:s)

```
sum = 0;
for (i=0; i<n; i++) {
    S1 sum = sum + a[i];
}
```

[Disegna le dipendenze fra le iterazioni del ciclo]



Allineamento dei cicli per rimuovere le dipendenze

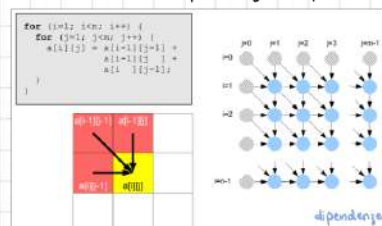


1. trasformare le dipendenze in dipendenze della stessa iterazione

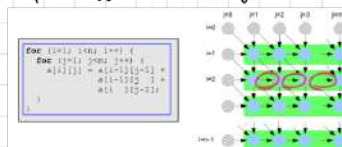
modificare l'indice dello S2 → deve compensare per non modificare l'algoritmo

- itero su un range diverso
- devo assegnare l'array che non rientrano più nel ciclo

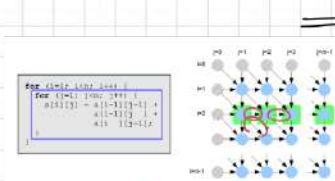
ESEMPIO Dipendenze complesse



Se parallelizzassimo il ciclo fuori



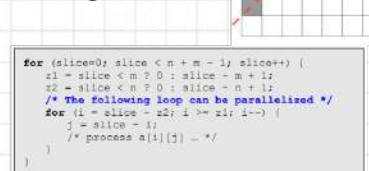
aumenterebbero le dipendenze fra le iterazioni



ma accadrebbe anche se parallelizzassimo il ciclo interno

Wavefront Sweep

iterare sulla matrice in diagonale



```
for (slice=0; slice < n + m - 1; slice++) {
    z1 = slice < m ? 0 : slice - m + 1;
    z2 = slice < m ? 0 : slice - m + 1;
    /* The following loop can be parallelized */
    for (i = slice; i < n; i++) {
        j = slice - i;
        /* process a[i][j] - */
    }
}
```

ESERCIZIO Può essere parallelizzato? Come?

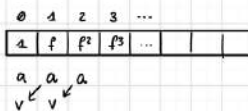
```
#define N some_big_number
double a[N];
const double f = ...;
double val = 1.0;
int i;

for (i=0; i<N; i++) {
    a[i] = val;
    val = val * f;
}
```

WAR su val

$a[0] = 1.0$

$val = 1.0 = f$



$v = f^i$
 for (i=0, i<N, i++) $\rightarrow$ Si è parallelizzabile
 $a[i] = \text{pow}(f, i)$

ESERCIZIO Se possono, quali cicli sono parallelizzabili? Perché?

```
#define N some_big_number
int s[N] = {1, 1, ..., 1};
double p[N] = {1, ..., 1};

/* Assume that function f() has no side effects */
for (int i=0; i<N; i++) {
    if (s[i]) {
        for (int j=0; j<N; j++) {
            if (s[j] && f(p[i], p[j])) {
                s[j] = 0;
            }
        }
    }
}
```

$\rightarrow$ Return true/false

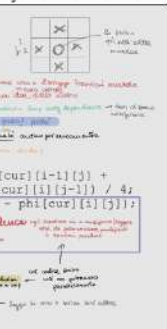
potrei parallelizzare il for interno

ESERCIZIO

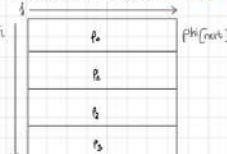
```
#define N 10000
double phi[2][N][N], maxdelta;
const double EPS = 1.0e-6;
int cur = 0, next = 1;

/* ...Initializations not shown... */

do {
    maxdelta = 0;
    for (int j=1; j<N-1; j++) {
        phi[next][i][j] = (phi[cur][i+1][j] + phi[cur][i-1][j] +
                          phi[cur][i][j+1] + phi[cur][i][j-1]) / 4;
        const double delta = fabs(phi[next][i][j] - phi[cur][i][j]);
        if (delta > maxdelta) {
            maxdelta = delta;
        }
    }
    /* exchange "cur" and "next" */
    const int tmp = cur; cur = next; next = tmp;
} while (maxdelta > EPS);
```



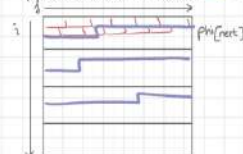
Parallelizzare solo $\rightarrow$ ciclo esterno



con schedule statico e chunk size di default

$\rightarrow$ divido e leggo per righe $\rightarrow$ ogni blocco un processore
 $\rightarrow$ aumentare la granularità non migliora di molto in questo caso

Parallelizzare entrambi $\rightarrow$ e $\rightarrow$ con collapse



posso usare una granularità + piccola delle righe
 $\rightarrow$ view visto come un unico array

La direttiva collapse ha un costo?

$\rightarrow$ il compilatore è come se avesse un solo ciclo su cui iterare
 in cui mappa i valori indici dei cicli interni

il compilatore potrebbe fare uso dell'operatore modulo

Nel nostro caso
 non c'è sbilanciamento del carico
 + corpo del ciclo semplice
 $\rightarrow$ collapse aggiunge overhead che ottiene un reale vantaggio

è utile x fare fine-tuning nella dimensione del chunk
 x migliori prestazioni

7 Performance Models

Work / Span Model

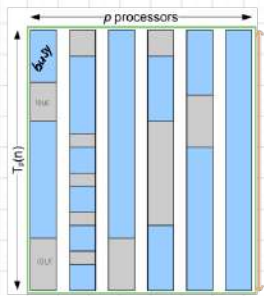
n input size $\equiv$ lato della matrice
 p num unità di esecuzione
 $T_p(n)$ wall-clock time del programma con p e $n \equiv$ cronometro del tempo di esecuzione

Work $T_1(n)$ wall-clock time con 1 unità di esecuzione
 $=$ num totale di operazioni che possono eseguire tutte le unità di esecuzione $\equiv$ costo di un algoritmo Seriale

Span $T_\infty(n)$ wall-clock time con p unità di esecuzione $\rightarrow$ non è detto che la computazione sia bilanciata
 $=$ num totale di operazioni che sono eseguite sulla catena più lunga di step sequenziali

Costo $p T_p(n) = \frac{\text{Span}}{\text{r'esecuzione}}$
 $=$ tempo totale speso da tutti i processori durante la computazione - sia fasi working che di waiting

Overhead $=$ Costo - Work
 $=$ quanto del tempo macchina è stato sprecato $\equiv$ quanto tempo di wait è stato usato



lunghezza delle barre blu $\propto$ al num di operazioni eseguite nello stesso tempo

Work $= T_1(n) \equiv \sum \text{lunghezze delle barre blu (busy)}$

Span $= T_\infty(n)$: tempo passato dall'inizio dell'esecuzione, alla fine

Costo $= p T_p(n)$: area dell'intero quadrato

Overhead $=$ costo - work $\equiv \sum \text{lunghezze delle barre grigie (idle)}$

parallel computation

Thm legge del Work $p T_p(n) \geq T_1(n)$

• overhead ≥ 0 per costruzione

Dim • $p T_p(n) - T_1(n) \geq 0$ da cui si ottiene la legge

p unità di esecuzione possono eseguire al massimo p operazioni allo stesso momento, in parallelo

Thm legge del Span $T_p(n) \geq T_\infty(n)$

il tempo di esecuzione di un programma parallelo che usa un num di unità di esecuzione p finito, non può essere minore del tempo di esecuzione con ∞ unità di esecuzione

Speedup $S(p,n) = \frac{T_1(n)}{T_p(n)}$: — dalla legge del Work $\rightarrow S(p,n) = \frac{T_1(n)}{T_p(n)} \leq \frac{p T_p(n)}{T_p(n)} = p$

$\Rightarrow$ Dunque $0 \leq \text{speedup} \leq p$, in particolare $\text{speedup} = p$ quando l'esecuzione ha overhead $= 0$, ovvero quando non ci sono mai periodi di idle

- * più è alto, meglio è
- * si calcola: fissato n , vario p
- * tipicamente sarà $S(p,n) < p$
 - x overhead sincronizzazione / scheduling
 - x algoritmi inefficienti

* **Speedup SUPERLINEARE** $S(p,n) > p \leftarrow$ Rarissimo

A ↑ num di unità di esecuzione $p \Rightarrow$ scompongo in + sotto problemi abbastanza $\Rightarrow$ Apparente INCREMENTO $\rightarrow$ ↓ working set su ogni p delle PRESTAZIONI

B con hardware eterogeneo si può notare che lo speedup reale $>>$ speedup teorico

C DECOMPOSIZIONE ESPLORATIVA — in base al tipo di composizione fatto

• vogliamo cercare un particolare valore in un albero non ordinato

↳ ESPLORO A LIVELLI per trovarlo

↳ posso dividere l'albero in + ESPLORAZIONI PARALLELE (a livelli) $\leftarrow$ + VELOCE ogni sezione ha meno nodi da esplorare

Strong Scaling Efficiency quanto il programma fa un uso ottimale dello Speedup

$$E(p,n) = \frac{S(p,n)}{p} = \frac{T_1(n)}{p T_p(n)}$$

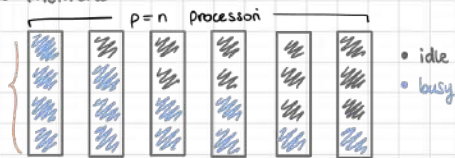
• dalla PROPIETA' $0 \leq S(p,n) \leq p$ — si ha $\rightarrow 0 \leq E(p,n) \leq 1$ — poiché $p T_p(n) = T_1(n) \leftrightarrow \text{overhead} = 0 \rightarrow \text{efficienza} = 1$

• avere una ALTA EFFICIENZA $\rightarrow$ si sta facendo un buon uso delle risorse

ESEMPIO Albero Sum-Reduce - come calcolare le metriche

```
float SumReduce( const float v[], int i, int j )
{
    if ( i > j )
        return 0;
    else if ( i == j )
        return v[i];
    else {
        const int m = (i + j) / 2;
        int s1, s2;
        #pragma omp task shared(s1)
        s1 = SumReduce(v, i, m);
        #pragma omp task shared(s2)
        s2 = SumReduce(v, m+1, j);
        #pragma omp taskwait
        return s1 + s2;
    }
}

/* Invocation */
float s;
float *v = ...
#pragma omp parallel
#pragma omp single
s = SumReduce(v, 0, n-1);
```



$$work = T_1(n) = 2^0 + 2^1 + 2^2 + \dots + n = O(n) \quad \text{! } \Sigma \text{ di tutte le lunghezze busy}$$

$$span = T_\infty(n) = O(\log n) \quad \text{altezza dell'albero}$$

$$Speedup: S(p,n) = \frac{T_1(n)}{T_p(n)} = \frac{O(n)}{O(\log n)} = O\left(\frac{n}{\log n}\right)$$

$$Efficienza E(p,n) = \frac{T_1(n)}{p T_p(n)} = \frac{O(n)}{p O(\log n)} = \frac{O(\frac{n}{\log n})}{p} \stackrel{p=n}{=} O\left(\frac{1}{\log n}\right) = O\left(\frac{1}{\log n}\right)$$

$$overhead = costo - work = O(n \log n) - O(n) = \max(O(n \log n), O(n)) = O(n \log n)$$

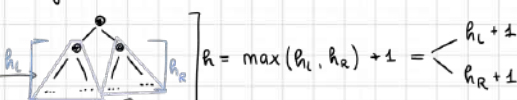
$$costo = p T_p(n) = n T_n(n) = O(n \log n)$$

Facciamo un'analisi più precisa con $p = \infty$ con le Master Thm

```
float SumReduce( const float v[], int i, int j )
{
    if ( i > j )
        return 0;
    else if ( i == j )
        return v[i];
    else {
        const int m = (i + j) / 2;
        int s1, s2;
        #pragma omp task shared(s1)
        s1 = SumReduce(v, i, m);
        #pragma omp task shared(s2)
        s2 = SumReduce(v, m+1, j);
        #pragma omp taskwait
        return s1 + s2;
    }
}

/* Invocation */
float s;
float *v = ...
#pragma omp parallel
#pragma omp single
s = SumReduce(v, 0, n-1);
```

ad ogni chiamata



⇒ ciascun SumReduce ha costo comp $O(\frac{n}{2})$

$$work = T_1(n) = 2 W(\frac{n}{2}) + O(1) = 2 O(\frac{n}{2}) + O(1) = \max(O(n), O(1)) = O(n)$$

$$span = T_\infty(n) = S(\frac{n}{2}) + O(1) = O(\log n) + O(1) = O(\log n)$$

$$Speedup = \frac{T_1(n)}{T_p(n)} = \frac{O(n)}{O(\log n)} = O\left(\frac{n}{\log n}\right)$$

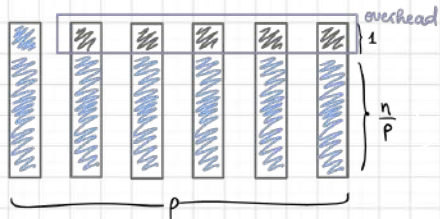
$$costo = p T_p(n) = p O(\log n) = O(p \log n)$$

$$overhead = costo - work = O(p \log n) - O(n) = O(p \log n)$$

$$efficienza = \frac{work}{costo} = \frac{O(n)}{O(p \log n)} = O\left(\frac{n}{p \log n}\right)$$

Se avessimo invece esattamente p processori,

dopo tutte le chiamate ricorsive si arriva a dividere il vettore lungo n in p blocchi



$$work = T_1(n) = p O(\frac{n}{p}) + O(1) = O(n)$$

$$span = T_\infty(n) = O(\frac{n}{p}) + O(1) = O(\frac{n}{p})$$

$$Speedup S(p,n) = \frac{T_1(n)}{T_p(n)} = \frac{O(n)}{O(\frac{n}{p})} = O\left(\frac{n p}{n}\right) = O(p)$$

$$Eff = \frac{Speedup}{p} = \frac{O(p)}{p} = 1 \quad \text{← migliore efficienza!}$$

$$costo = p T_p(n) = p O(\frac{n}{p}) = O(n)$$

$$overhead = costo - work = O(n) - O(n) = O(n)$$

Sono ottimi!

Stima dello Span $T_p(n)$

- fissato l'INPUT SIZE = n ← grande a sufficienza in modo che il tempo di esecuzione dell'algo non sia troppo piccolo
- definiamo $T_p(n)$ il Wall-Clock Time di un programma parallelo con $p = 1 \dots P$ processori
- da $T_p(n)$ si calcolano → Speedup $S(p,n) = \frac{T_1(n)}{T_p(n)}$
→ Efficiency $E(p,n) = \frac{S(p,n)}{p}$

è importante avere il Wall-Clock Time escluso I/O Time!

```
#include "hpc.h" // WCT è non
other #includes here...
...
double start, finish;
/* read input; this should not be time consuming */

start = hpc_gettime(); // omp | mpi | cuda
/* actual code that we want to time */
finish = hpc_gettime();
```

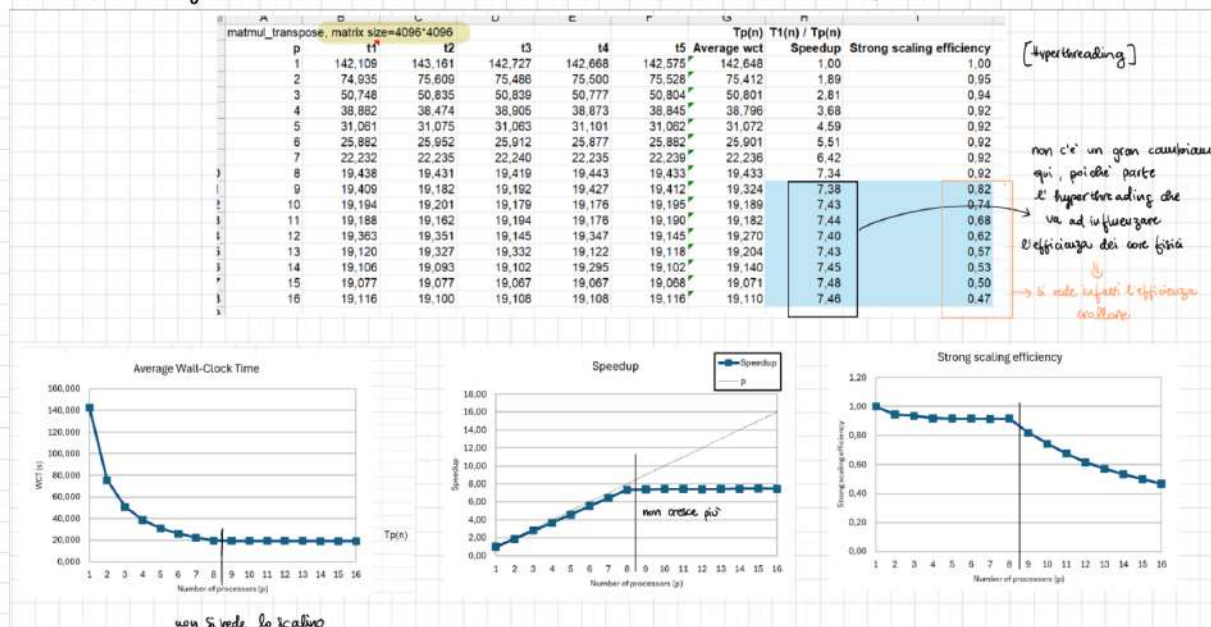
← Sfrutto la libreria dinamica del prof

> vale sia per OpenMP, MPI e CUDA

usa

- Using OpenMP: `omp_get_wtime()`
- Using MPI: `MPI_Wtime()`
- Generic solution: `clock_gettime()`
- **Wrong solution:** `clock()` → non fa quello che ci aspettiamo in multithread

ESEMPIO Confronto delle metriche su Moltiplicazione tra MATRICI 4096 * 4096 TRASPOSTE



Tipi di Efficienza

STRONG SCALING

aumentare il numero di processori p
con la stessa dimensione totale del problema

↓ work per singolo proc.
↑ num di p in uso

work costante

× RIDURRE IL TEMPO DI ESECUZIONE TOTALE
aggiungendo + processori

$$E(p,n) = \frac{S(p,n)}{p} = \frac{T_1(n)}{p T_p(n)}$$

WEAK SCALING

aumentare il numero di processori p
con lo stesso work per ognuno

↓

↑ quantità totale di work
↑ num di p in uso

× RISOLVERE PROBLEMI PIÙ GRANDI
NELLO STESSO INTERVALLO DI TEMPO

$$\text{Weak } E(p) = \frac{T_1(n_1)}{T_p(n_p)}$$

dove n_p è la dimensione di input in
proporzione ai processori

$$f(n_p, p) = f(n_1, 1)$$

ovvero i due work relativi al singolo processore
devono essere uguali

1. DEVO CALCOLARE IL WORK [TOTALE]
2. applico le variabili con i corrispettivi $n_1, n_p, 1, p$
3. RISOLVO L'EQUAZIONE e RICOVERO $n_p = \dots$

Legge di Amdahl

per quanto si possa parallelizzare resta sempre una parte seriale che limita il tempo

sia T_{serial} il tempo totale sequenziale, e' diviso in

- α FRAZIONE SERIALE : % di codice che non può essere parallelizzata
- $1-\alpha$ FRAZIONE PARALLELA : % di codice rimanente che può essere divisa equamente tra i processori

allora si ha $T_{\text{parallel}}(p) = \alpha T_{\text{serial}} + \frac{(1-\alpha) T_{\text{serial}}}{p}$

resta invariato
indipendentemente da p

si riduce sempre
di più con tanti p → TEMPO a 0

OSSERVAZIONE

$$\text{Lo Speedup } S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{T_{\text{serial}}}{\alpha T_{\text{serial}} + \frac{(1-\alpha) T_{\text{serial}}}{p}} = \frac{1}{\alpha + \frac{(1-\alpha)}{p}}$$

in particolare se $p \rightarrow \infty$ allora $\text{Speedup}_{\text{max}} = \frac{1}{\alpha}$

Dunque se si ha un programma con 5% di codice seriale $\rightarrow \alpha = 0.05$

Lo speedup teorico non può mai superare $S_{\text{max}} = \frac{1}{0.05} = 20$, ovvero che anche con potenza di calcolo illimitata il programma andrà al massimo 20 volte più veloce.

8 Programmazione memoria distribuita con MPI

message passing

modello di programmazione principale per supercomputer e cluster

MPI

libreria usata con tipici linguaggi sequenziali (es. fortran, C, C++)

basato su **Single Program Multiple Data (SPMD)** (= lo stesso programma è eseguito da P processi, ognuno in base al ID può avere un percorso di esecuzione ≠)

garantisce isolamento di area degli indirizzi → **no race conditions**, ma **possibili errori di comunicazione**

→ espone il modello di esecuzione e forza il programmatore a pensare localmente, **ottimo per le performance**

→ **aumentano complessità e il codice**

! Tutte le operazioni di comunicazione e sincronizzazione richiedono chiamate di subroutine → **No variabili condivise**



- ★ ORGANIZZAZIONE dei PROCESSI
 - raggruppati in **groups**
 - groups + contesto = comunicatore
 - ID = rank e groups w/ comunicatore

- ★ DATI : < indirizzo, valore, type >
 - PREDEFINITO [MPI_INT, MPI_DOUBLE ...]
 - ARRAY di tipi
 - STRIDE block
 - ARRAY indirizzato
 - STRUCT

disarivano il layout che devono avere in memoria

MPI Datatype	C datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_LONG_LONG	signed long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED_INT	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	

- ★ TAG = intero definito dall'utente, contenuto nel messaggio inviato x aiutare il processo dest a identificare il messaggio a filtrare

MPI_ANY_TAG : no tag

MPI_Init(&argc, &argv)
MPI_Finalize()

funzioni per aprire/chiedere un blocco MPI
↳ Solo all'interno di queste si possono fare chiamate MPI

MPI_COMM_WORLD

comunicatore predefinito con gruppo = { tutti processi }

MPI_Comm_size() → num di processi

MPI_Comm_rank() → ID del processo diamante $0 \leq \text{rank} < \text{size}$

MPI_Send() (buf, count, datatype, dest, tag, comm)

SEND BLOCCANTE

MESSAGE BUFFER

→ ritorna ⇒ dati inviati e buffer possono essere riutilizzati
il messaggio potrebbe non essere ancora stato ricevuto

MPI_Recv() (buf, count, datatype, source, tag, comm, status)

RECV BLOCCANTE

aspetta fino a quando non matcha source e tag in un messaggio

count = n° ELEMENTI da inviare

dest = rank del processo target | MPI_PROC_NULL ⇒ Send non ha effetto

tag = tag del messaggio [salvo sempre 0 per noi]

source = rank del processo specificato in comm | MPI_ANY_SOURCE

tag = da matchare | MPI_ANY_TAG

count — check : se ricevo > count ⇒ ERROR

≤ count ⇒ OK

status = ulteriori informazioni sul msg | MPI_STATUS_IGNORE se non sono richieste

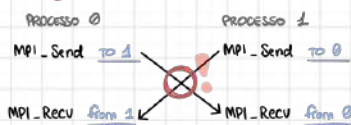
```

MPI_Status {
    int MPI_SOURCE
    int MPI_TAG
    int MPI_ERROR
}
    
```

MPI_Get_count(&status, datatype, &count)

controlla effettivamente quanti elementi ci sono in status e scrive il num nell'indirizzo di mem di count

PROBLEMA ! Deadlock



→ ORDINARE le operazioni in modo che coppie Send-Recv matchino

→ usare Send/Recv non bloccanti

MPI_Sendrecv()

MPI_Isend(*start, count, datatype, dest, tag, comm, MPI_Request *req)

Send non bloccante → asincrono: termina quando lo status ha dei dati ⇒ viene aggiornato

→ RITORNA immediatamente

identificatore univoco di questa richiesta

MPI_Irecv (*start, count, datatype, source, tag, comm, MPI_Request *req)

Recv non bloccante → avvia operazione di RICEZIONE in BACKGROUND, restituendo il controllo al programma immediatamente

fin quando il msg non è ricevuto
è VIETATO LEGGERE/MODIFICARE il buffer start → Ma quando finisce?!

x l'operazione di recv è ancora in corso
sfrutto req con

ferma il programma e ASPETTA il completamento dell'operazione
i DATI del msg erano arrivati? No → li aspetto
Sì → sblocco tutto

MPI_Wait(&req, &status)

Bloccante

MPI_Test(&req, &flag, &status)

Non blocc

chiede rapido con RETURN flag = 1 || 0
recv TERMINATA recv ATTIVA

ESISTONO due VARIANTI

x gestire l'ARRAY di req CONTEMPORANEAMENTE

MPI_WaitTest any | all | some (-)

Quando finisce req viene automaticamente svuotata da MPI_REQUEST_NULL

MPI_Abort(comm, err) termina in modo "carino" tutte le funzioni MPI sul comunicatore comm
→ RETURN del codice di err

Comunicazioni collettive

comm più efficienti e veloci delle point-to-point
→ ottenendo lo stesso risultato

Bulk synchronous

COME dovete funzionare in APPLICAZIONI REALI → i processi lavorano ad ondate

LOCAL COMPUTATION ogni processo si chiude nel suo guscio e fa processi locali ← Nessuno comunica con altri

GLOBAL COMPUTATION o SINCRONIZZAZIONE tutti i processi si fermano e mettono insieme i risultati x aggiornare la visione globale del problema

MPI_Barrier ()
quando un processo arriva a questa direttiva si BLOCCA e FERMA in questo punto
→ quando TUTTI i processi si sono fermati → la BARRIERA CROLLA e TUTTI i PROCESSI vengono LIBERATI CONTEMPORANEAMENTE

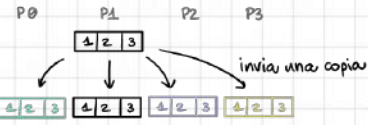
MPI_Bcast (buf, count, datatype, source, comm)

dati da INIZIARE / RICEVERE
ID proc MITT

il proc SOURCE deve COPIARE e DISTRIBUIRE i dati del buffer

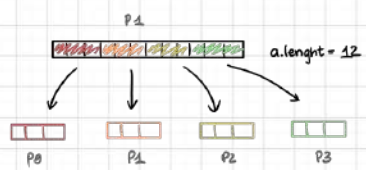
è DINAMICA: in base al ruolo del processo invocante [MITT | DEST]

⇒ Tutti i processi interessati devono ESEGUIRLA — altrimenti → Deadlock



MPI_Scatter (sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, source, comm)

disperdere / sparpagliare



il proc SOURCE ha un array che DIVIDE e DISTRIBUISCE ad ogni processo

↳ num proc * sendcount = array.length

sendcount = prendo l'arrayone e lo invio 3 blocchi alla volta a # processo

recvcount = num di elementi che devo ricevere nel buffer locale

è DINAMICA: in base al ruolo del processo invocante [MITT | DEST]

⇒ Tutti i processi interessati devono ESEGUIRLA — altrimenti → Deadlock

MITT = Source Sendbuf → PRENDE → recvbuf (COPIA)
DEST usa solo recvbuf, Sendbuf viene IGNORATO → può essere NULL

MPI_Gather (sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, dest, comm)

operazione INVERSA alla Scatter
con stesse caratteristiche ma con ruoli invertiti [MITT | DEST]

ESISTONO due VARIANTI

SCATTERV e GATHERV

- 1. msg size diverse ✓
- 2. buchi tra i msg ✓
- 3. ordine di distribuzione qualsiasi ✓

x superare limite di blocchi uguali

⇒ grazie a Sendcount in forma ARRAY

MPI_Allgather (sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, dest, comm)

Gather che broadcasta il risultato dell'arrayone

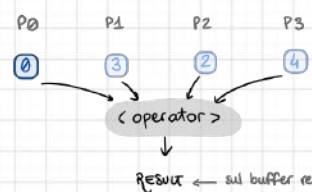
MPI_Reduce (sendbuf, recvbuf, count, datatype, operator, dest, comm)

racoglie tutti i dati e li combina con l'OPERATORE matematico e SALVA il risultato sul buffer del DEST

è DINAMICA: in base al ruolo del processo invocante [MITT | DEST]

⇒ Tutti i processi interessati devono ESEGUIRLA — altrimenti → Deadlock

! con array [count > 1] fa riduzione elem. x elem
Tutti gli i-esimi con gli i-esimi



MPI_Allreduce (sendbuf, recvbuf, count, datatype, operator, comm)

combina i dati di tutti i processi con operatore e DISTRIBUISCE il risultato finale a tutti i processi

MPI_Alltoall (sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, comm)

devono coincidere

ogni processo fa una SCATTER verso gli altri e allo stesso tempo una GATHER dei pezzi ricevuti

MPI_Scan (sendbuf, recvbuf, count, datatype, operator, comm)

calcola la SOMMA PARZIALE in ordine di ID (rank)

il processo 0 riceve nel recvbuf il risultato della REDUCE (operator, Sendbuf con 0 ≤ ID ≤ 5)

Operation	Value	Meaning
MPI_MAX	Maximum	one value from
MPI_MIN	Minimum	
MPI_SUM	Sum	
MPI_PROD	Product	
MPI_LAND	Logical AND	
MPI_BAND	Bitwise AND	rank & 11 < 12
MPI_BOR	Logical OR	
MPI_BOR	Bitwise OR	rank & 11 < 12
MPI_EXOR	Logical exclusive OR	
MPI_EXOR	Bitwise exclusive OR	
MPI_MAXLOC	Maximum and location of maximum	
MPI_MINLOC	Minimum and location of minimum	

9 CUDA Programming

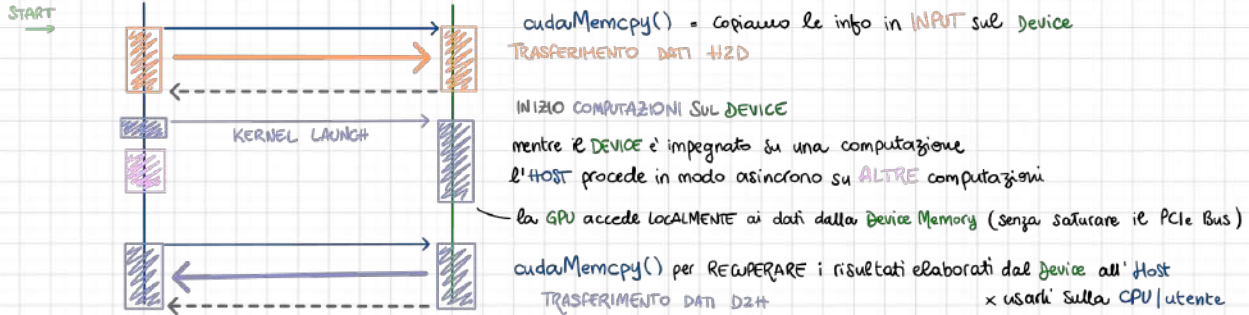
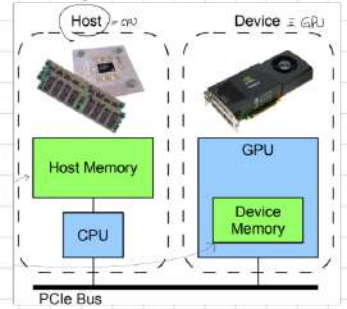
CUDA = Compute Unified Device Architecture

- Espone un modello di programmazione Data-Parallel (= applicato su dati diversi)
- E' proprietario — alternativa open source → OpenCL [non molto diffusa]
- DIVIDE il programma in → CPU-PROGRAM $\in$ {operazioni I/O, interazione utente ...}
- GPU-PROGRAM $\in$ computazioni eseguite dalla GPU

cercando INTERAZIONE tra i 2 device (host e device)



Dispositivi di calcolo



THREAD = unità di lavoro minima, tutti vengono eseguiti dalla stessa funzione del kernel

- Raggruppati per WARPs di 32 threads ← GPU $\approx$ CPU SIMD

↑
unità hardware di raggruppamento (su cui non possiamo agire o intervenire)

BLOCCO = insieme di thread assegnati a Stream Multiprocessor (CORE fisico della GPU)

- Strutturati 1D array | 2D matrici | 3D cubi → x mappare comodamente i thread con coordin.
- LIMITE hardware: MAX 1024 threads / blocco [basato su WARPs]

GRID = intero spazio di calcolo per un singolo lancio di kernel ← dove e cosa mando quando mando in esecuzione la GPU

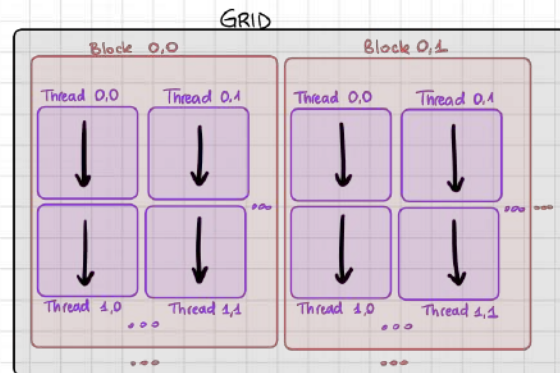
- Strutturati 1D array | 2D matrici | 3D cubi

Calcolare l'indice globale del thread

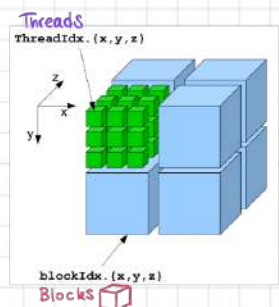
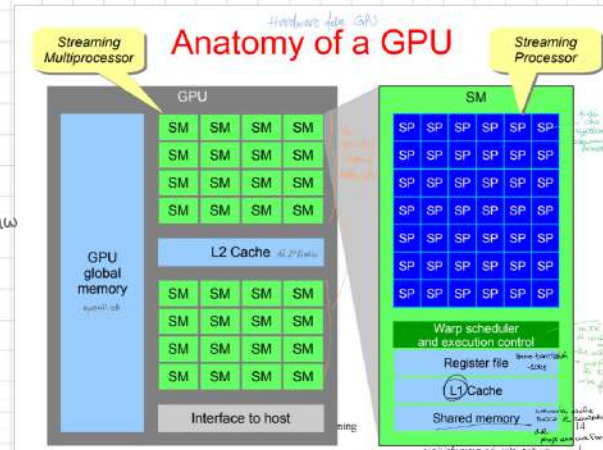
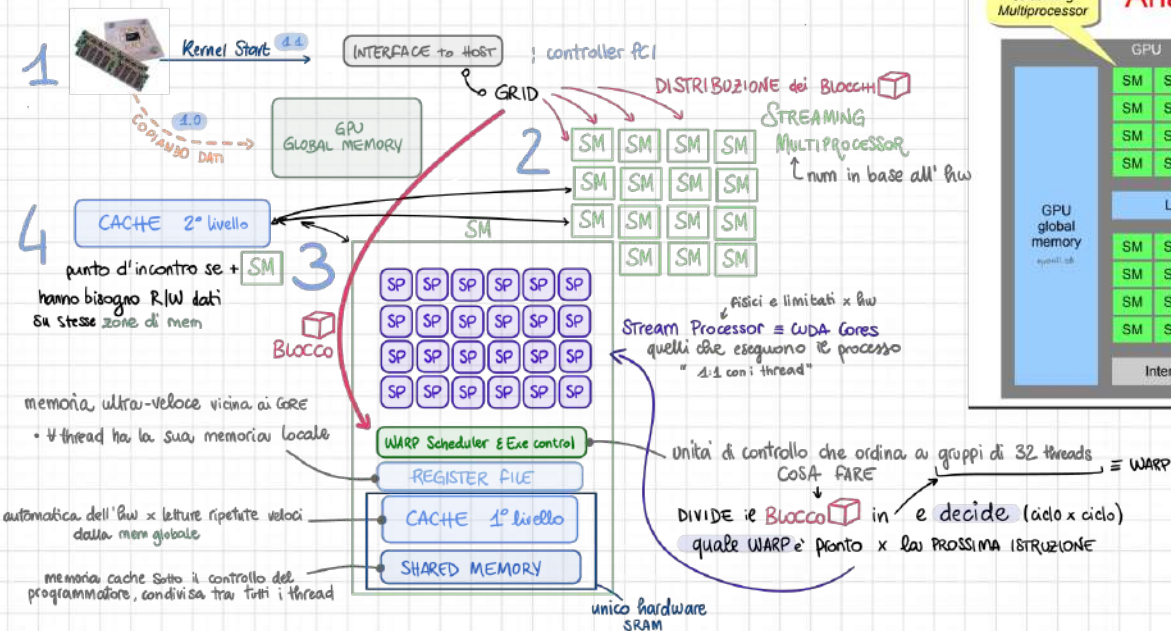
→ thread 0,0 su Block 0,1 ← obiettivo 0

1. prima di block 0,1 c'è block 0,0 ⇒ che ha blockDim.x $\equiv$ num di thread contenuti in un blocco
2. quanti blocchi ci lasciamo alle spalle? blockIdx.x = 1 ⇒ ho solo 1 blocco alle spalle

⇒ ID-globale $x = (\text{blockDim.x} * \text{blockIdx.x}) + \text{threadIdx.x}$



Flusso hardware: COME VIAGGIA IL CALCOLO nella GPU?



thread-block EVOLUZIONE

- * al lancio del kernel, si possono creare molti più block degli SM fisici disponibili
- * l'ordine dell'associazione block → SM è NON-DETERMINISTICO → Quale si libera prima?
- ⇒ thread di block diversi NON possono ASPETTARSI o SINCRONIZZARSI (non è detto che esistano)

MEMORY COALESCING

richiesta di accesso in memoria da + thread, in locazioni adiacenti → un'unica MAXI-RICHIESTA alla mem → consegnata ai threads (la smistano loro)

SIMT

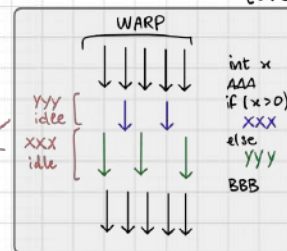
Single Instruction Multiple Threads = i thread di un WARP condividono l'istruzione ma operano su DATI PRIVATI (registri)

WARP DIVERGENCE

= quando i thread dello stesso WARP prendono strade diverse in un if/else

Spegliamo le loro ALU
x EVITARE di seguire dati sbagliati sui registri sbagliati

GPU sfrutta una EXECUTION MASK
x ABILITARE/DISABILITARE i bit 110 x i thread che sono RUNNING/IDLE in al ciclo succ



0 Hello World

```

/* cuda-hello0.cu */
#include <stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
    
```

```

$ nvcc cuda-hello0.cu
$ ./a.out
Hello World!
    
```

non sfrutta/usa la GPU, e' eseguito sulla CPU

```

/* cuda-hello1.cu */
#include <stdio.h>
__global__ void mykernel(void) {}

int main(void)
{
    mykernel<<<1,1>>>>();
    printf("Hello World!\n");
    return 0;
}
    
```

GPU

CPU

— global — funzione sul Device chiamata sull'Host
% deve RITORNARE void

STR <<< n.Block, Thread.x.Block >>> (PARAMS...)
indica il Kernel Launch

che è ASINCRONO
= il controllo torna SUBITO alla CPU

La CPU deve sincronizzarsi prima di usare i risultati

cudaMemcpy()

blocca la CPU finché il trasferimento dati non è terminato

cudaMemcpyAsync()

Scrive sulla CPU in modo ASINCRONO → senza bloccare la CPU

cudaDeviceSynchronize()

SERIE PARAMS

fa una memcpy ma tenendo bloccata la CPU finché tutte le CUDA calls precedenti hanno finito

PROBLEMA: Numero di elementi non divisibile per la blockDim

A Kernel: Boundary Check

```

__global__ void add(int *a, int *b, int *c, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < n) { // mio globalIdx < dim max?
        c[index] = a[index] + b[index];
    }
}
    
```

introduce Warp Divergence x evitare crash

B Host: Kernel Launch

Quanti blocchi lanciare nel file principale CPU

```
add<<<(N + blockDim-1)/blockDim, blockDim>>>(d_a, d_b, d_c, N);
```

Le CPU non scalano più in frequenza a causa dei limiti termici. Per accelerare i calcoli scientifici siamo passati alle GPU, che sacrificano le logiche di controllo e le cache giganti per massimizzare il num di ALU sul chip. Questo sposta l'onere al programmatore: il parallelismo non è più un optional hw, ma deve essere scritto esplicitamente nel codice

SHARED MEMORY

un unico grande spazio di RAM globale su cui possono RW tutti i processori liberamente → x comunicare usano variabili comuni

DISTRIBUTED MEMORY

ogni processore ha la sua RAM privata e personale → x comunicare devono mandare messaggi sulla rete

Qualificatori dello spazio di esecuzione della funzione

-- global --

ESEGUITO SU

Device

CHIAMATO DA

Host

identifica un Kernel | deve sempre restituire void | si lancia con <<<NBlock, NThr>>>

-- device --

Device

Device

sono le sotto-funzioni ausiliare che si usano nel kernel | Non si possono chiamare fuori

-- host --

Host

Host

funzione C classica, se non specificato è il comportamento predefinito

* Si possono ACCORPARE → il compilatore genererà 2 versioni della stessa funzione compilata per CPU/GPU

! NO FUNZIONI RICORSIVE sul Device → x consumerebbe troppi registri/stack & thread

! NO VARIABILI static in funzioni def sul Device

! NO PARAMETRI CON NUMERO VARIABILI di args → x

x modello SIMT avrebbe stack configurati ≠ nei thread ⇒ No Mem Coalescing
↳ x tutto il WARP devono essere configurati uguali

`cudaMalloc((void**)& d.a, size)` → RITORNA `cudaError_t` x dire se l'allocazione è andata a buon fine o se GPU ha finito la mem
⇒ il puntatore allocato è nella posizione di mem passata

`cudaMemcpy(dest, source, numbyte, dir)` sposta i dati sul bus PCIe (in base alla direzione) su CPU | GPU
DIREZIONE = [dove stanno andando i dati] `cudaMemcpyHostToDevice` | `cudaMemcpyDeviceToHost`

`cudaFree(d.a)` libera la memoria della GP

